

1. INTRODUCTION

Solar energy is one of the most promising renewable resources, offering a clean and sustainable alternative to fossil fuels. Accurate estimation of solar radiation is essential for designing and optimizing solar energy systems such as photovoltaic and solar thermal plants. However, solar radiation at the Earth's surface is influenced by several atmospheric and meteorological factors—including temperature, cloud cover, wind speed, wind direction, and pressure—making reliable hourly estimation challenging. The difficulty is further increased by natural variability and the limited availability of ground-based measurement stations.

Advances in data-driven techniques have positioned machine learning (ML) as an effective solution for solar radiation forecasting. ML models can uncover complex, nonlinear relationships between meteorological variables and solar irradiance without depending on detailed physical atmospheric models. In particular, Random Forest regression is well-suited for this task due to its ability to handle nonlinear patterns, feature interactions, and outliers, while maintaining strong resistance to overfitting.

This project focuses on building a machine learning model to predict hourly solar radiation using surface meteorological data. The dataset is sourced from the NASA POWER database, which provides global parameters such as temperature, precipitation, wind speed, wind direction, and atmospheric pressure. The workflow involves preprocessing the data to handle missing values, outliers, and cyclic features (hour and month), followed by training, testing, and evaluating the model.

Accurate solar radiation prediction supports renewable energy planning, grid stability, and environmental monitoring. It helps solar plant operators maximize energy output, reduce reliance on conventional power sources, and support sustainable energy strategies. By combining historical meteorological data with machine learning, this project offers a practical and efficient approach for hourly solar radiation estimation.

In summary, the work demonstrates the potential of using meteorological data and ML techniques to develop reliable solar radiation prediction models, contributing to more efficient use of solar energy resources.

2. SUPPORTING LITERATURE

Literature Review

Paper 1 – Colucci R, Mahgoub I. *Solar Irradiance Prediction with Ensemble Learning Method as Input for Battery Operation Optimization*. In 2024 IEEE Texas Power and Energy Conference (TPEC) 2024 Feb 12 (pp. 1-6). IEEE.

The paper titled “Solar Irradiance Prediction Utilizing an Ensemble Learning Approach for Optimizing Battery Operations” tackles the issue of precisely forecasting solar irradiance to improve the functionality of Battery Energy Storage Systems (BESS) within microgrid settings. As the world shifts from fossil fuels to renewable energy sources such as solar and wind, the authors point out that the variability and high costs associated with installations pose significant challenges to widespread implementation. Reliable solar irradiance forecasting can be crucial in optimizing the charge-discharge cycles of batteries, prolonging their lifespan, reducing operating expenses, and facilitating more effective energy management in microgrids for both residential and commercial purposes.

To model solar irradiance, the authors utilize five well-known machine learning algorithms: Random Forest (RF), Extreme Gradient Boosting (XGBR), Support Vector Regression (SVR), Kernel Ridge Regression (KRR), and Linear Regression (LR). These models are based on meteorological and environmental factors obtained from the NASA POWER satellite database. The dataset used spans six years (2015-2020) and includes variables such as temperature, relative humidity, precipitation, wind speed, clearness index, surface skin temperature, and atmospheric pressure. Data from 2015 to 2019 is employed for training, while 2020 is set aside for testing and validation.

Additionally, the study implements three ensemble learning techniques—Voting, Stacking, and Bagging—by combining the most effective base models to improve predictive performance. SVR was notably excluded from this ensemble phase due to its comparatively poor solo outcomes. The effectiveness of all models was evaluated using four standard metrics: Mean Squared Error (MSE), Root Mean Squared Error (RMSE), Mean Absolute Error (MAE), and the R^2 score.

Among the individual models, Random Forest yielded the most favorable results with an R^2 score of 0.9513, surpassing XGBR (0.9510), KRR (0.9392), and LR (0.9430). In contrast,

SVR significantly lagged behind with an R^2 of 0.5817. The ensemble methods exhibited slight but consistent enhancements over the best standalone models, with the Stacking Regressor delivering the highest overall performance ($R^2 = 0.9546$), closely followed by Voting (0.9536) and Bagging (0.9511). When compared to earlier studies that applied deep learning techniques like LSTM, GRU, CNN, and attention-based models ($R^2 \approx 0.69$ – 0.71) on the same NASA POWER dataset, the proposed ensemble machine learning techniques demonstrated a notable improvement in performance.

The study concludes that ensemble learning methods, particularly stacking, can substantially increase the accuracy of solar irradiance predictions when compared to individual machine learning models and previous deep learning approaches utilizing the same dataset. The authors suggest future work focused on devising an algorithm for battery operation optimization that directly incorporates the predicted irradiance values, considering the strong correlation between solar irradiance and photovoltaic (PV) power output.

TITLE	Solar Irradiance Prediction with Ensemble Learning Method as Input for Battery Operation Optimization
AREA OF WORK	The paper focuses on applying machine learning and ensemble learning algorithms to predict solar irradiance for optimizing Battery Energy Storage Systems (BESS) in microgrid environments. The work highlights how accurate irradiance forecasting can improve renewable energy utilization, reduce costs, and enhance battery lifespan.
DATASET	NASA POWER satellite-based dataset (2015–2020), including meteorological parameters: temperature at 2m, relative humidity, precipitation, wind speed at 10m, clearness index, earth skin temperature, and surface pressure.
METHODOLOGY /STRATEGY	The study uses five years of historical data (2015–2019) for training and one year (2020) for validation. Data preprocessing involves feature selection from meteorological parameters. Five machine learning algorithms—Random Forest, Extreme Gradient Boosting, Support Vector Regression, Kernel Ridge Regression, and Linear Regression—are trained and evaluated using error metrics (MSE, RMSE, MAE, R^2). To enhance prediction accuracy, three ensemble methods (Voting, Stacking, Bagging) are applied, combining the strengths of the

	individual models.
ALGORITHM	• Random Forest Regressor • Extreme Gradient Boosting Regressor (XGBR) • Support Vector Regression (SVR) • Kernel Ridge Regression (KRR) • Linear Regression (LR) • Ensemble methods: Voting, Stacking, Bagging
RESULT/PRECISION	• Best standalone: Random Forest ($R^2 = 0.9513$). • Best ensemble: Stacking Regressor ($R^2 = 0.9546$). • Outperformed prior deep learning models on the same dataset (e.g., LSTM, GRU, CNN, Attention, with $R^2 \approx 0.69\text{--}0.71$).
ADVANTAGES	The study demonstrates that ensemble learning models, especially stacking, achieve superior accuracy compared to both standalone machine learning and deep learning approaches. It validates the applicability of NASA POWER data for irradiance prediction, integrates multiple regression algorithms for robustness, and shows direct relevance to microgrid and battery optimization applications.
FUTURE PROPOSAL	The authors propose developing a full-scale battery operation optimization algorithm using predicted irradiance as a key input. Future research will also explore refining ensemble strategies, incorporating real-time forecasting, and extending the framework to optimize PV system sizing and overall microgrid cost-efficiency.

Table 2.1 Summary of Paper 1

Paper 2 – Imam AA, Abusorrah A, Seedahmed MM, Marzband M. *Accurate forecasting of global horizontal irradiance in Saudi Arabia: a comparative study of machine learning predictive models and feature selection techniques*. Mathematics. 2024 Aug 22;12(16):2600.

This paper provides an in-depth comparative examination of forecasting Global Horizontal Irradiance (GHI) utilizing machine learning regression models, concentrating on northern Saudi Arabia—a region selected for its significant solar radiation potential and its key position in the country's renewable energy development, especially with the establishment of the 300 MW Sakaka Solar PV Power Plant. The necessity for precise GHI predictions is

underscored as a crucial element for optimizing solar PV systems, efficiently operating energy storage, and ensuring stable grid integration. The authors utilized a comprehensive dataset from the NASA POWER database, covering the years 1984 to 2022, which encompassed GHI alongside meteorological and solar parameters such as Direct Normal Irradiance (DNI), temperature, wind speed at two different heights, humidity, dew point, surface pressure, clearness index, precipitation, and surface albedo. Data preparation entailed cleaning, min-max normalization, and splitting the data into training and testing sets in an 80–20 ratio.

Six regression models—Artificial Neural Network (ANN), Decision Tree (DT), Elastic Net (EN), Linear Regression (LR), Random Forest (RF), and Support Vector Regression (SVR)—were created and evaluated. To enhance the inputs, three feature selection methods (forward, backward, and exhaustive search) were employed. Forward selection determined that temperature, clearness index, and DNI were the most significant predictors, while backward and exhaustive searches also incorporated relative humidity. Despite surface pressure showing a strong correlation with GHI, it was omitted by all methods, indicating redundancy with other selected features.

The effectiveness of the models was gauged using R^2 , MSE, RMSE, MAPE, and MAE. The Decision Tree stood out as the leading performer with perfect fit on training data ($R^2 = 1.0$, all error metrics = 0), followed closely by RF ($R^2 = 0.9987$) and ANN ($R^2 = 0.9976$), both of which demonstrated extremely low error rates. LR ($R^2 = 0.9986$) and SVR ($R^2 = 0.9878$) also yielded strong performances but exhibited slightly larger discrepancies, while EN ($R^2 = 0.8396$) significantly underperformed. Cross-validation using k-fold and shuffle split methods validated the resilience of the top models, although the authors caution that DT models may be susceptible to overfitting in varying conditions, making RF a preferred option due to superior generalization. The results of feature selection offered minimal improvements for most models but greatly boosted SVR's performance, indicating its vulnerability to unrelated inputs.

The research concludes that DT and RF are the most appropriate algorithms for predicting GHI in areas with similar climatic conditions, providing high precision, dependability, and practicality for extensive solar forecasting. These findings present crucial insights for renewable energy planning, leading to more accurate PV generation projections and improved integration of solar resources within national energy initiatives.

TITLE	Accurate Forecasting of Global Horizontal Irradiance in Saudi Arabia: A Comparative Study of Machine Learning Predictive Models and Feature Selection Techniques
AREA OF WORK	The paper focuses on evaluating the performance of six machine learning regression models in forecasting Global Horizontal Irradiance (GHI) for northern Saudi Arabia, aiming to improve solar energy planning, PV system optimization, and grid integration.
DATASET	NASA POWER database (1984–2022), containing GHI and multiple meteorological variables: Direct Normal Irradiance (DNI), temperature, wind speed (10 m, 50 m), relative humidity, dew point, surface pressure, clearness index, precipitation, and surface albedo.
METHODOLOGY/STRATEGY	Historical data was preprocessed using min–max normalization and split into training (80%) and testing (20%). Six regression models were trained: ANN, DT, EN, LR, RF, and SVR. Feature selection (forward, backward, exhaustive search) was applied to identify the most influential predictors. Model performance was compared using R^2 , MSE, RMSE, MAPE, and MAE, with additional validation through k-fold and shuffle split cross-validation.
ALGORITHM	• Artificial Neural Network (ANN) • Decision Tree (DT) • Elastic Net (EN) • Linear Regression (LR) • Random Forest (RF) • Support Vector Regression (SVR)
RESULT/PRECISION	Decision Tree achieved the highest performance ($R^2 = 1.0$, zero error) but showed signs of overfitting. Random Forest ($R^2 = 0.9987$) offered more reliable accuracy, followed by ANN ($R^2 = 0.9976$) and LR ($R^2 = 0.9986$). SVR performed moderately ($R^2 = 0.9878$, improved with feature selection), while Elastic Net was weakest ($R^2 = 0.8396$).
	The study provides a robust comparative analysis of classical ML

ADVANTAGES	regression models for GHI prediction, integrates advanced feature selection methods, and validates results with cross-validation. It demonstrates that Decision Tree and Random Forest are highly reliable for solar irradiance forecasting in regions with high solar potential.
FUTURE PROPOSAL	Future research may focus on expanding the models to real-time forecasting applications, integrating satellite-based and ground-based hybrid datasets, testing models across diverse climatic zones, and combining ensemble strategies to improve generalizability and reduce risks of overfitting.

Table 2.2 Summary of Paper 2

Paper 3 – Demir V. *Evaluation of Solar Radiation Prediction Models Using AI: A Performance Comparison in the High-Potential Region of Konya, Türkiye*. Atmosphere. 2025 Mar 30;16(4):398.

The research article titled “Evaluation of Solar Radiation Prediction Models Using AI: A Performance Comparison in the High-Potential Region of Konya, Türkiye” examines the effectiveness of different machine learning and deep learning techniques in forecasting solar radiation based on extensive meteorological data. This study tackles the worldwide need for precise solar radiation estimation, which is vital for enhancing renewable energy systems, especially in regions with high potential like Konya, Turkey. The research leverages 39 years of hydro-meteorological data (1984–2022) sourced from NASA/POWER and MERRA-2 datasets, utilizing temperature, relative humidity, precipitation, and wind speed as input variables, while solar radiation (kWh/m²/day) is set as the target output.

The approach included training and evaluating 14 different models, encompassing deep learning structures (LSTM, Bi-LSTM, GRU, Bi-GRU), boosting techniques (LSBoost, XGBoost), ensemble models (Bagging, Random Forest), kernel-based methods (SVM with linear, polynomial, and Gaussian kernels), and variations of neural networks (MLANN, RBANN, GRNN). Preprocessing of data, correlation assessment, and parameter optimization were carried out to enhance model reliability. The dataset was divided into 75% for training and 25% for testing, employing evaluation metrics such as Root Mean Square Error (RMSE), Mean Absolute Error (MAE), and the coefficient of determination (R²).

The findings revealed that deep learning models notably outperformed traditional machine learning techniques during the testing phase, with LSTM yielding the highest predictive accuracy (RMSE = 0.34, MAE = 0.27, $R^2 = 0.97$), closely succeeded by Bi-LSTM (RMSE = 0.37, MAE = 0.31, $R^2 = 0.97$) and GRU (RMSE = 0.41, MAE = 0.33, $R^2 = 0.97$). Although LSBoost demonstrated remarkable performance during training (RMSE = 0.08, MAE = 0.06, $R^2 = 0.998$), it experienced overfitting, resulting in a considerable drop in testing accuracy (RMSE = 0.99, $R^2 = 0.77$). Among the classical models, Random Forest (RMSE = 0.77, $R^2 = 0.86$) and Bagging (RMSE = 0.80, $R^2 = 0.85$) showed moderate levels of accuracy, whereas the SVM variants consistently performed at a lower level.

The study concludes that while LSTM-based deep learning models are superior overall, tree-based ML methods like Random Forest and Bagging remain practical, reliable, and computationally efficient alternatives for solar radiation prediction. Boosting methods require careful regularization to prevent overfitting, and kernel-based methods such as SVM may be less suitable for long-term irradiance forecasting. The authors recommend exploring additional atmospheric parameters, such as aerosols, to further improve ML model accuracy.

TITLE	Evaluation of Solar Radiation Prediction Models Using AI: A Performance Comparison in the High-Potential Region of Konya, Türkiye
AREA OF WORK	The paper focuses on predicting solar radiation using classical machine learning and deep learning approaches, comparing their effectiveness in a region with high solar energy potential.
DATASET	NASA/POWER and MERRA-2 datasets (1984–2022) containing temperature ($^{\circ}\text{C}$), relative humidity (%), precipitation (mm/day), and wind speed (m/s) as input variables, with solar radiation ($\text{kWh/m}^2/\text{day}$) as the target.
METHODOLOGY/STRATEGY	The study applied data preprocessing, correlation analysis, and model training using a 75:25 train–test split. Performance was evaluated using RMSE, MAE, and R^2 . The comparison centered on classical machine learning models, including ensemble methods (Random Forest, Bagging), boosting algorithms (XGBoost, LSBoost),

	kernel-based approaches (SVM with different kernels), and neural network variants (GRNN, MLANN, RBANN). Overfitting control methods such as dropout and early stopping were used.
ALGORITHM	LSBoost, XGBoost, Bagging, Random Forest, Support Vector Machines , GRNN, MLANN, RBANN
RESULT/PRECISION	Random Forest ($R^2 = 0.86$, RMSE = 0.77) and Bagging ($R^2 = 0.85$, RMSE = 0.80) provided the most reliable performance among ML models. LSBoost achieved very high training accuracy ($R^2 = 0.998$) but dropped to $R^2 = 0.77$ on testing, indicating overfitting. XGBoost showed moderate accuracy, while SVM variants consistently performed weakest ($R^2 \approx 0.80$). Neural network variants such as GRNN, MLANN, and RBANN yielded mixed but less competitive results compared to ensemble methods.
ADVANTAGES	The paper compares a wide spectrum of ML and DL algorithms using long-term meteorological data, providing a robust benchmark for solar radiation forecasting. It shows that LSTM-based models capture temporal dependencies effectively, while tree-based models provide stability. The findings can guide energy planners in optimizing solar farms and renewable energy integration.
FUTURE PROPOSAL	The authors suggest incorporating additional atmospheric variables (e.g., aerosols, cloud properties) for improved accuracy, exploring hybrid ML-DL frameworks, and extending the model for real-time solar energy forecasting and integration into smart grid systems.

Table 2.3 Summary of Paper 3

2.1. Literature Review Summary

TITLE	SUMMARY
Colucci R, Mahgoub I. <i>Solar Irradiance Prediction with Ensemble Learning Method as Input for Battery Operation Optimization</i> . In 2024 IEEE Texas Power and Energy Conference (TPEC) 2024 Feb 12 (pp. 1-6). IEEE.	This study forecasts solar irradiance to improve battery energy storage system (BESS) operation in microgrid environments. Using NASA/POWER data (2015–2020), it analyzed predictors such as temperature, humidity, precipitation, wind speed, pressure, and clearness index. Five ML algorithms—Random Forest, XGBoost, Support Vector Regression (SVR), Kernel Ridge Regression, and Linear Regression—were evaluated. Among single models, Random Forest performed the best, while ensemble approaches (stacking, voting, bagging) further enhanced accuracy, with stacking yielding the overall highest results. The findings show that precise irradiance forecasting supports extended battery lifespan, reduced costs, and more efficient renewable microgrid management.
Imam AA, Abusorrah A, Seedahmed MM, Marzband M. <i>Accurate forecasting of global horizontal irradiance in Saudi Arabia: a comparative study of machine learning predictive models and feature selection techniques</i> . Mathematics. 2024 Aug 22;12(16):2600.	This work compared six ML models—Decision Tree (DT), Random Forest (RF), Support Vector Regression (SVR), Linear Regression (LR), Elastic Net (EN), and Artificial Neural Network (ANN)—for forecasting global horizontal irradiance (GHI) in northern Saudi Arabia using NASA/POWER data (1984–2022). Features included direct normal irradiance, temperature, humidity, wind speed, pressure, precipitation, and albedo. Feature selection (forward, backward, exhaustive) improved performance. While DT achieved the highest reported accuracy ($R^2 = 1.0$), it is prone to overfitting; RF provided a more stable and generalizable solution. The study highlights that robust ensemble methods outperform complex or overly simple models when features are carefully

	selected.
Demir V. <i>Evaluation of Solar Radiation Prediction Models Using AI: A Performance Comparison in the High-Potential Region of Konya, Türkiye.</i> Atmosphere. 2025 Mar 30;16(4):398.	This study assessed multiple solar radiation prediction models using NASA/POWER data (1984–2022) from Konya, Türkiye. Machine learning algorithms evaluated included Random Forest, XGBoost, Bagging, and Support Vector Machine (SVM). Meteorological variables used were temperature, humidity, precipitation, and wind speed. Results showed that tree-based ensemble methods (RF, XGBoost, Bagging) consistently achieved strong performance, demonstrating that classical ML approaches provide accurate and efficient solutions for solar forecasting. Although deep learning methods were also tested, this work confirms that ensemble machine learning remains a reliable and practical choice for solar radiation prediction.

Table 2.4 Overall Summary of all paper

2.2. Findings and Proposals

The initial study reviewed by Colucci R and Mahgoub I (2024) examined the prediction of solar irradiance using ensemble learning to enhance battery performance within microgrid systems. The authors evaluated five machine learning techniques: Random Forest (RF), Extreme Gradient Boosting (XGBoost), Support Vector Regression (SVR), Kernel Ridge Regression (KRR), and Linear Regression (LR). Their findings indicated that ensemble methods, especially stacking, increased predictive accuracy compared to standalone models. The primary takeaway from this research is that integrating multiple machine learning algorithms improves the reliability of solar irradiance forecasting, making it more applicable for practical uses such as managing battery energy storage systems (BESS) and reducing costs in microgrids.

The second study by Imam AA, Abusorrah A, Seedahmed MMA, and Marzband M (2024) presented a comparative analysis of machine learning predictive models and feature selection methods for predicting global horizontal irradiance (GHI) in Saudi Arabia. The study

examined six algorithms: Artificial Neural Network (ANN), Decision Tree (DT), Elastic Net (EN), Linear Regression (LR), Random Forest (RF), and Support Vector Regression (SVR). Although ANN was part of the investigation, the best results emerged from Decision Tree models, which demonstrated strong predictive capabilities when utilized alongside feature selection techniques such as forward, backward, and exhaustive searches. The insight from this study is that even simpler machine learning approaches like DT can perform exceptionally well when significant meteorological features (temperature, humidity, wind, precipitation, and pressure) are thoughtfully chosen and refined.

The third research effort by Demir V (2025) evaluated the effectiveness of different solar radiation prediction models in the sun-rich area of Konya, Türkiye. While the paper considered both deep learning and traditional ML methods, the results emphasized that ensemble tree-based algorithms like Random Forest, Bagging, and XGBoost yielded high predictive accuracy along with statistical robustness. These algorithms exhibited strong performance while demanding fewer resources compared to more complex methods. The conclusion from this study is that ensemble machine learning techniques continue to be dependable and practical for forecasting solar radiation, striking a balance between accuracy, interpretability, and resource efficiency.

The studies reviewed together support a comparative evaluation of Random Forest, XGBoost, and Support Vector Regression for solar radiation prediction using hourly NASA/POWER data for one location. Each algorithm presents distinct advantages: RF is sturdy and minimizes overfitting, XGBoost offers superior predictive abilities through boosting, and SVR is adept at modeling nonlinear patterns in weather data. The forthcoming work will train and assess these models based on selected meteorological factors—temperature, relative humidity, precipitation, wind speed, wind direction, and surface pressure—utilizing performance measures such as R^2 , MAE, RMSE, and MAPE. The best-performing model will be integrated into a Graphical User Interface (GUI), allowing users to input meteorological data and receive immediate solar radiation predictions, serving as a valuable decision-support tool for renewable energy planning and battery optimization.

3. SYSTEM ANALYSIS

The Solar Radiation Dataset, sourced from NASA/POWER (Prediction of Worldwide Energy Resources), provides high-resolution hourly meteorological and solar parameters, which are crucial for predicting surface-level solar irradiance. This dataset enables a comprehensive analysis of atmospheric and environmental factors affecting solar energy availability and variability.

Dataset Link: <https://power.larc.nasa.gov/data-access-viewer/>

Size: The dataset spans from January 2015 to August 2025, covering more than 90,000 hourly observations for a single geographic location (Latitude: 10.1004, Longitude: 76.357). Each record contains hourly values of solar radiation, weather, and atmospheric variables.

Data Structure: It contains multiple features such as temperature at 2 meters (T2M), relative humidity (RH2M), dew point temperature (T2MDEW), surface pressure (PS), precipitation (PRECTOTCORR), wind speed (WS10M), wind direction (WD10M), and insolation clearness index (ALLSKY_KT), along with the target variable surface shortwave downward irradiance (ALLSKY_SFC_SW_DWN). These features support a variety of analyses, from understanding atmospheric influences on solar radiation to building machine learning regression models for accurate solar radiation prediction.

Sparsity: The dataset is generally complete and reliable, as it is derived from NASA/POWER reanalysis sources. However, certain variables contain missing values, represented by -999, during nighttime or when measurements are unavailable. These missing values need to be handled during preprocessing through techniques like removal, imputation, or feature engineering. Overall, the dataset maintains good coverage across years, hours, and meteorological parameters, providing sufficient density for robust solar radiation prediction using machine learning models.

3.1. Analysis of dataset

3.1.1. About the dataset

a) Dataset Overview

The Solar Radiation Dataset, sourced from NASA/POWER, contains hourly records from 2015 to 2025 and includes multiple meteorological and atmospheric parameters that influence

solar radiation. The dataset is specifically designed for predictive modeling of solar radiation levels (ALLSKY_SFC_SW_DWN), which is the target variable.

- Total records: ~90,000+ (hourly data for multiple years)
- Total Features: 12 independent variables + 1 target variable
- Data Types: All features are numerical (float or integer).
- Missing Values: Present in some parameters (e.g., ALLSKY_KT contains -999 placeholders for missing values)

Feature Categories:

1. Temporal Features

- YEAR: Calendar year (2015–2025)
- MO: Month (1–12)
- DY: Day of month (1–31)
- HR: Hour of the day (0–23)

2. Meteorological & Atmospheric Parameters

- T2M: Temperature at 2 meters (°C)
- RH2M: Relative Humidity at 2 meters (%)
- PRECTOTCORR: Precipitation Corrected (mm/hour)
- WS10M: Wind Speed at 10 meters (m/s)
- PS: Surface Pressure (kPa)
- T2MDEW: Dew/Frost Point at 2 meters (°C)
- ALLSKY_KT: All Sky Insolation Clearness Index (dimensionless)
- WD10M: Wind Direction at 10 meters (degrees)

3. Target Variable:

- ALLSKY_SFC_SW_DWN: Surface Shortwave Downward Irradiance (Wh/m²)

```
print("Shape of dataset:",df.shape)
print("\nData types\n",df.dtypes)

Shape of dataset: (93096, 13)

Data types
YEAR          int64
MO            int64
DY            int64
HR            int64
ALLSKY_SFC_SW_DWN  float64
T2M           float64
RH2M          float64
PRECTOTCORR    float64
WS10M         float64
PS            float64
T2MDEW        float64
ALLSKY_KT     float64
WD10M         float64
dtype: object
```

Fig 3.1 Result of shape and datatype of dataset

b) Dataset sample records

This displays the first 10 records to get an initial look at the dataset's structure and values.

```
df = pd.read_csv(file_path, skiprows=17)
print(df.head(10))
```

Fig 3.2 Loading and displaying first ten rows

	YEAR	MO	DY	HR	ALLSKY_SFC_SW_DWN	T2M	RH2M	PRECTOTCORR	WS10M	\
0	2015	1	1	0	0.00	25.24	88.43	4.52	3.39	
1	2015	1	1	1	0.00	25.03	88.84	5.64	3.42	
2	2015	1	1	2	0.00	24.84	89.13	5.49	3.31	
3	2015	1	1	3	0.00	24.70	89.45	4.70	3.04	
4	2015	1	1	4	0.00	24.57	89.67	5.00	2.80	
5	2015	1	1	5	0.00	24.45	89.76	4.01	2.55	
6	2015	1	1	6	23.42	24.65	89.44	2.85	2.33	
7	2015	1	1	7	164.57	25.40	86.54	3.34	2.63	
8	2015	1	1	8	348.42	26.18	82.17	0.00	2.88	
9	2015	1	1	9	445.15	27.31	75.36	0.00	3.18	
	PS	T2MDEW	ALLSKY_KT	WD10M						
0	100.27	23.21	-999.00	277.6						
1	100.21	23.09	-999.00	277.2						
2	100.18	22.95	-999.00	278.5						
3	100.22	22.87	-999.00	280.8						
4	100.25	22.78	-999.00	282.0						
5	100.33	22.69	-999.00	282.9						
6	100.42	22.82	0.26	284.7						
7	100.52	23.01	0.41	286.8						
8	100.58	22.92	0.51	288.0						
9	100.58	22.60	0.49	285.0						

Fig 3.3 First ten record of the dataset

c) Summary statistics

The function `df[num_cols].describe().T` is used to generate summary statistics for numeric columns in a DataFrame. It calculates key metrics such as count, mean, standard deviation, minimum and maximum values, and the 25th, 50th, and 75th percentiles.

```
df = df.replace(-999, np.nan)
num_cols = df.select_dtypes(include=['int64', 'float64']).columns
summary_stats = df[num_cols].describe().T
print(summary_stats)
```

	count	mean	std	min	25%	\
YEAR	93096.0	2019.820830	3.070994	2015.00	2017.00	
MO	93096.0	6.390822	3.428495	1.00	3.00	
DY	93096.0	15.697860	8.800371	1.00	8.00	
HR	93096.0	11.500000	6.922224	0.00	5.75	
ALLSKY_SFC_SW_DWN	91272.0	212.139598	285.814380	0.00	0.00	
T2M	93024.0	27.660533	2.317248	19.73	26.00	
RH2M	93024.0	78.669538	13.757996	19.45	72.69	
PRECTOTCORR	93024.0	7.472025	14.619828	0.00	0.08	
WS10M	93024.0	2.912059	1.451922	0.01	1.85	
PS	93024.0	100.409948	0.230552	99.21	100.26	
T2MDEW	93024.0	23.284884	2.241069	6.98	22.62	
ALLSKY_KT	46484.0	0.463597	0.162767	0.00	0.34	
WD10M	93024.0	236.547178	86.285023	0.00	218.70	
	50%	75%	max			
YEAR	2020.00	2022.00	2025.00			
MO	6.00	9.00	12.00			
DY	16.00	23.00	31.00			
HR	11.50	17.25	23.00			
ALLSKY_SFC_SW_DWN	5.03	427.73	1041.32			
T2M	27.30	28.78	37.49			
RH2M	82.61	88.79	97.76			
PRECTOTCORR	1.82	8.53	295.72			
WS10M	2.76	3.80	14.36			
PS	100.41	100.56	101.30			
T2MDEW	23.91	24.68	27.38			
ALLSKY_KT	0.48	0.59	0.79			
WD10M	267.80	292.50	359.80			

Fig 3.4 Summary statistics of numeric columns

3.1.2. Exploratory Data Analysis

a) Distribution of the Target Variable

Purpose: The histogram of solar radiation (`ALLSKY_SFC_SW_DWN`) is plotted to understand how radiation values are distributed across the dataset, check for skewness, and identify dominant ranges. This helps reveal whether the dataset is balanced across day and night observations.

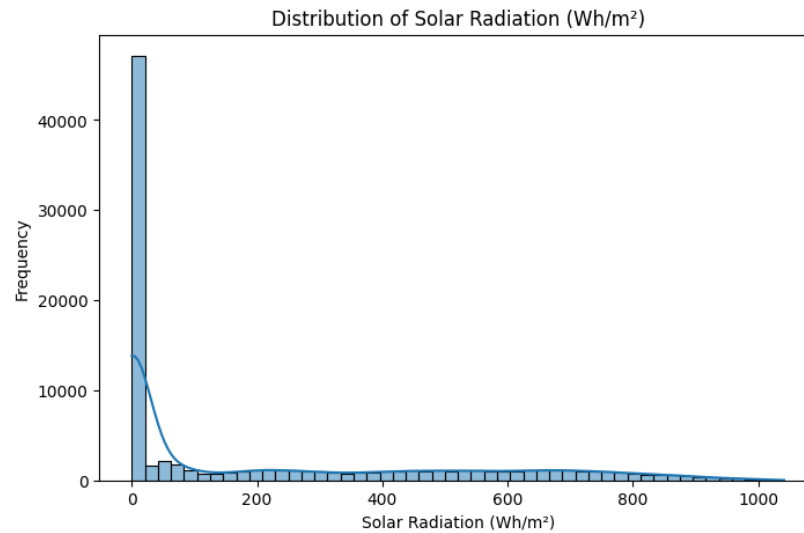


Fig 3.5 Target Variable Distribution

b) Time Series Visualization

Purpose: A time series plot is used to examine how solar radiation varies over time, highlighting daily and seasonal trends. This provides insights into periodic patterns that affect prediction.

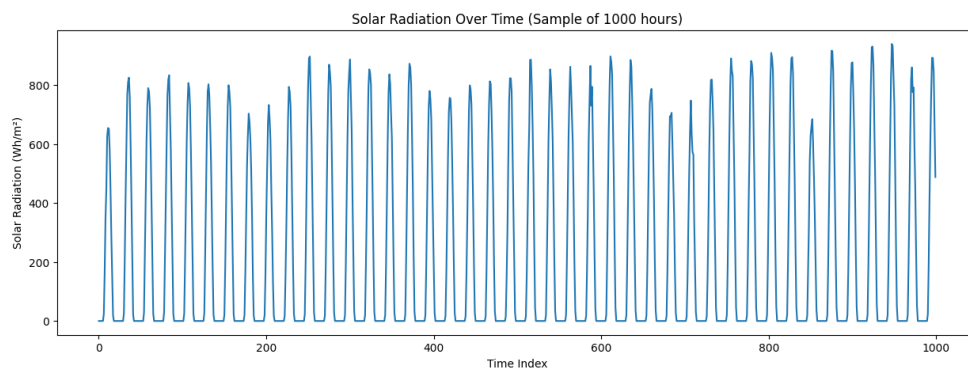


Fig 3.6 Time Series Plot

c) Correlation Heatmap

Purpose: A correlation heatmap is generated to measure relationships between solar radiation and meteorological features, and to detect multicollinearity among predictors. This helps in feature selection and understanding variable importance.

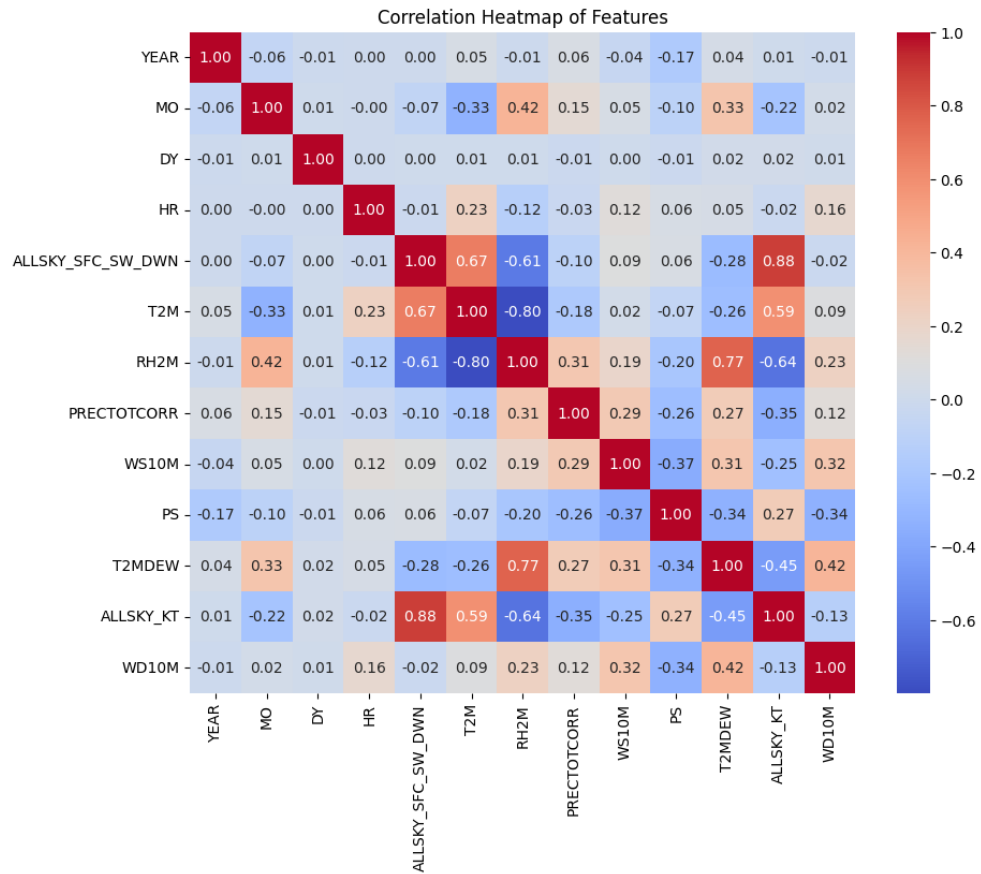


Fig 3.7 Correlation Matrices

d) Boxplots by Time Features

Purpose: Boxplots are created by hour of the day and by month to capture how solar radiation varies with time. This helps identify cyclical and seasonal dependencies.

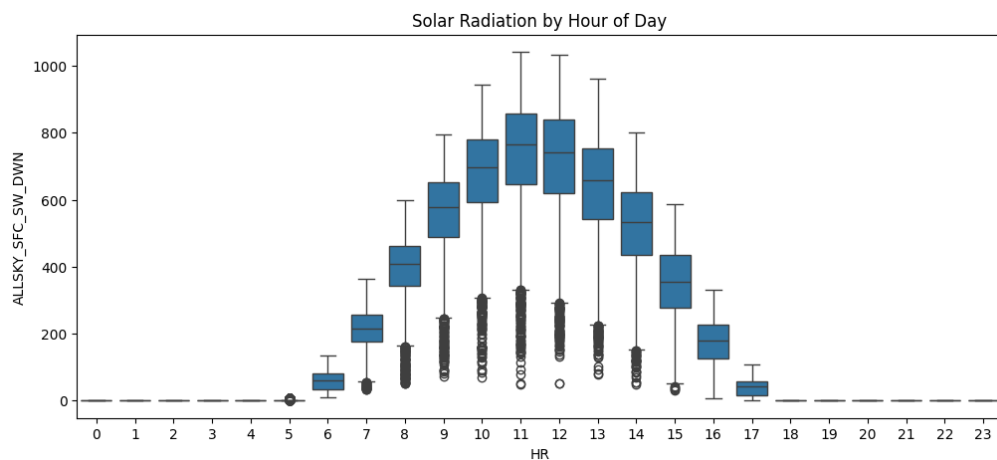


Fig 3.8 Solar radiation pattern based on hour

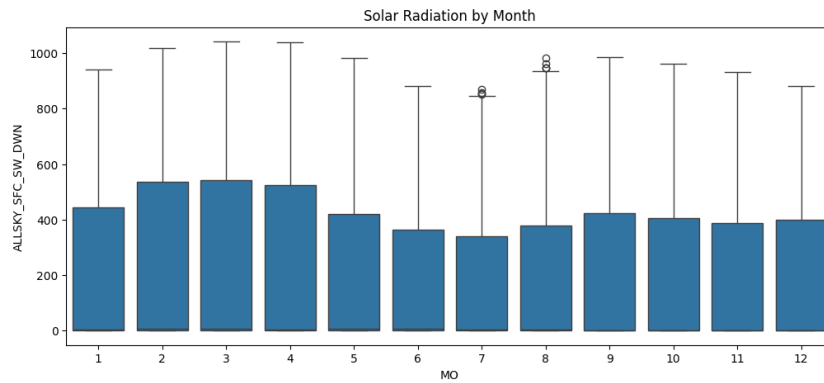


Fig 3.9 Solar radiation pattern based on month

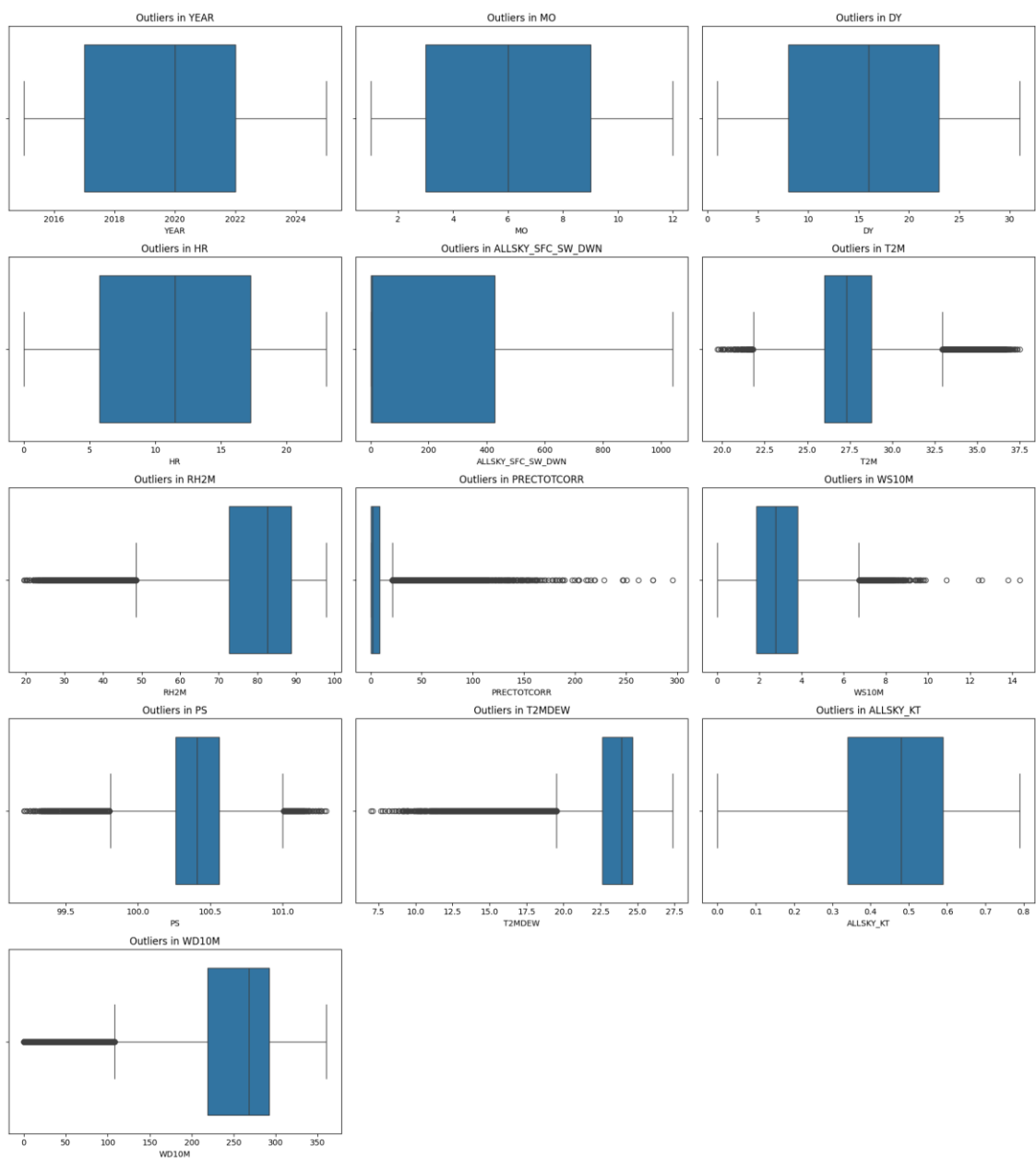


Fig 3.10 Boxplot for each column

Boxplots reveal that the temporal features (YEAR, MO, DY, HR) exhibit a uniform distribution without any outliers. The target variable (ALLSKY_SFC_SW_DWN) is positively skewed, predominantly reflecting low to moderate radiation levels, with occasional high peaks observed under clear-sky conditions.

Among the meteorological variables:

- T2M, RH2M, WS10M, and PRECTOTCORR display natural outliers as a result of weather fluctuations (temperature extremes, dry/humid conditions, sudden rainfall, and strong winds).
- PS and WD10M demonstrate stability with minimal fluctuation.
- ALLSKY_KT and T2MDEW are fairly well-distributed, featuring only a few mild outliers.

3.1.3. Insights and conclusions of Exploratory Data Analysis

The histogram of solar radiation shows that most of the recorded values are concentrated around zero, which represents nighttime hours when no solar radiation is received. A smaller portion of the data is distributed across higher values, indicating daytime peaks, which makes the target variable highly skewed.

The time series plot of solar radiation confirms the expected diurnal cycle, where radiation increases sharply in the morning, peaks around midday, and falls back to zero during the night. This cyclical behavior repeats consistently across hours, showing the strong temporal dependency of solar radiation.

The correlation heatmap highlights that solar radiation has a strong positive correlation with temperature (T2M) and clearness index (ALLSKY_KT), while it is negatively correlated with relative humidity (RH2M). Other features, such as surface pressure (PS) and wind speed (WS10M), show weaker correlations, suggesting they have less direct impact on solar radiation.

The boxplot by hour of the day clearly illustrates that solar radiation is zero during night hours and gradually rises from early morning, reaching its peak between 11:00 AM and 2:00 PM, before declining towards evening. This confirms the strong influence of the time of day on radiation levels.

The boxplot by month shows seasonal variations in radiation, with generally higher values during summer months and lower values during winter months. This indicates that month-based features are also important predictors in capturing seasonal effects on solar radiation. Boxplot display natural outliers as a result of weather fluctuations (temperature extremes, dry/humid conditions, sudden rainfall, and strong winds).

Overall, the analysis demonstrates that solar radiation is strongly dependent on temporal factors (hour and month) and key meteorological parameters (temperature, humidity, clearness index), validating their importance for predictive modeling.

3.2. Data Preprocessing

3.2.1. Data Cleaning

a) Checking for missing values

The dataset was checked for missing values across all columns using the `isna().sum()` function. The results revealed that most meteorological features had 72 missing entries (Temperature, Relative Humidity, Precipitation, Wind Speed, Pressure, Dew Point, Wind Direction). The target variable, `ALLSKY_SFC_SW_DWN`, contained 1,824 missing values, while the `ALLSKY_KT` (Clearness Index) feature had a significant number of missing entries (46,612 records).

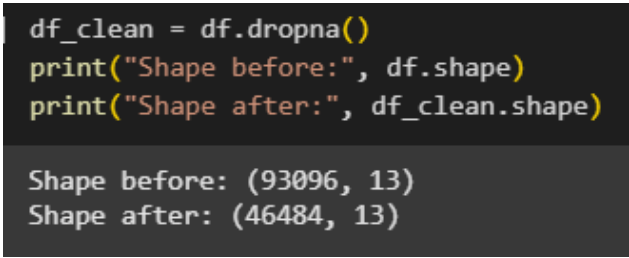
```
print("Missing values:\n",df.isna().sum())

Missing values:
YEAR                0
MO                  0
DY                  0
HR                  0
ALLSKY_SFC_SW_DWN  1824
T2M                  72
RH2M                 72
PRECTOTCORR         72
WS10M                72
PS                   72
T2MDEW              72
ALLSKY_KT           46612
WD10M                72
dtype: int64
```

Fig 3.11 Missing values

b) Removal of missing values

The dataset was analyzed for any missing values, and rows with NaN entries were eliminated utilizing the `dropna()` function. Originally, the dataset comprised 93,096 records across 13 columns. After eliminating all rows with missing values, the dataset was condensed to 46,484 complete records. This process ensured that only reliable and consistent data points remained for subsequent analysis and model development, thus reducing potential biases or errors stemming from incomplete records.



```
df_clean = df.dropna()
print("Shape before:", df.shape)
print("Shape after:", df_clean.shape)

Shape before: (93096, 13)
Shape after: (46484, 13)
```

Fig 3.12 Shape after handling missing values

c) Dropping columns

In order to enhance the dataset for modeling purposes, features that were either irrelevant or highly correlated were eliminated. The columns YEAR and DY were discarded because they do not offer significant predictive value concerning solar irradiance. Moreover, RH2M (Relative Humidity at 2 meters) and T2MDEW (Dew Point Temperature) were also excluded due to their high multicollinearity with other meteorological variables, which could adversely impact the performance of the model. Furthermore, the feature ALLSKY_KT (Clear Sky Index) was removed to avoid redundancy, as it is derived from solar radiation and can lead to data leakage during prediction. The final dataset included 8 features: MO, HR, ALLSKY_SFC_SW_DWN (target), T2M, PRECTOTCORR, WS10M, PS, and WD10M, thereby maintaining a balance between predictive capability and minimized redundancy.

```
df_processed = df_clean.drop(columns=['YEAR', 'DY', 'RH2M', 'T2MDEW', 'ALLSKY_KT'])

print("Final Features:")
for ftr in df_processed:
    print(ftr)

Final Features:
MO
HR
ALLSKY_SFC_SW_DWN
T2M
PRECTOTCORR
WS10M
PS
WD10M
```

Fig 3.13 Features after dropping columns

d) Outlier handling

This step centers on addressing outliers to maintain data consistency and enhance model accuracy. For PRECTOTCORR (precipitation) and WS10M (wind speed), a logarithmic transformation utilizing `np.log1p()` was implemented. This technique effectively diminishes the influence of highly skewed extreme values while retaining the relative disparities among observations.

For T2M (temperature) and PS (surface pressure), the lower and upper boundaries were established using the 1st and 99th percentiles. Values outside these boundaries were clipped using the `clip()` function to prevent the model from being affected by unrealistic extremes.

```
# Precototcorr
df_clean['PRECTOTCORR'] = np.log1p(df_clean['PRECTOTCORR'])

#T2M
lower_bound = df_clean['T2M'].quantile(0.01)
upper_bound = df_clean['T2M'].quantile(0.99)
df_clean['T2M'] = df_clean['T2M'].clip(lower=lower_bound, upper=upper_bound)

#PS
lower_bound = df_clean['PS'].quantile(0.01)
upper_bound = df_clean['PS'].quantile(0.99)
df_clean['PS'] = df_clean['PS'].clip(lower=lower_bound, upper=upper_bound)

#WS10M
df_clean['WS10M'] = np.log1p(df_clean['WS10M'])
```

Fig 3.14 Outlier handling

3.2.2. Feature Extraction

a) Handling cyclic features

Some numerical features in the dataset—hour (HR), month (MO), and wind direction (WD10M)—exhibit cyclical or repetitive patterns. Using these as straightforward numeric values can mislead the model since values like 0 and 23 (for hour) or 1 and 12 (for month) may appear far apart numerically but are actually close in concept.

To accurately reflect these connections, each cyclic feature was transformed into sine and cosine components:

- Hour (HR): Encoded with sine and cosine functions based on a 24-hour cycle.
- Month (MO): Encoded with a periodicity of 12 months to account for seasonal variations.
- Wind Direction (WD10M): Converted from degrees to radians and then represented as sine and cosine to maintain directional consistency.

This transformation preserves the cyclic characteristics of time and direction data, enabling the machine learning model to recognize periodic patterns in solar radiation without creating artificial discontinuities.

```
df_processed['HR_sin'] = np.sin(2 * np.pi * df_processed['HR'] / 24)
df_processed['HR_cos'] = np.cos(2 * np.pi * df_processed['HR'] / 24)

df_processed['MO_sin'] = np.sin(2 * np.pi * df_processed['MO'] / 12)
df_processed['MO_cos'] = np.cos(2 * np.pi * df_processed['MO'] / 12)

df_processed['WD_sin'] = np.sin(np.deg2rad(df_processed['WD10M']))
df_processed['WD_cos'] = np.cos(np.deg2rad(df_processed['WD10M']))

df_processed = df_processed.drop(columns=['HR'])
df_processed = df_processed.drop(columns=['MO'])
df_processed = df_processed.drop(columns=['WD10M'])
```

Fig 3.15 Handling cyclic features

3.2.3. Analysis of Feature Variables and Class Variables

a) Correlation Analysis

To explore the linear relationships between meteorological factors and solar radiation, a correlation heatmap was created. The findings revealed that air temperature (T2M) showed a

moderate positive correlation of 0.58. Conversely, variables like PS (Surface Pressure), MO (Month), and PRECTOTCORR (Precipitation) demonstrated weak negative or low correlations. Parameters related to wind (WS10M, WD10M) and HR (Hour) had minimal or negligible correlation with solar radiation. This analysis informed the feature selection process, emphasizing the most pertinent predictors while pinpointing features with limited predictive capability.

```
plt.figure(figsize=(10,8))
sns.heatmap(df_processed.corr(), annot=True, cmap="coolwarm", center=0 ,fmt=".2f")
plt.title("Correlation Heatmap of Features")
plt.show()

# Correlation with target variable
target_corr = df_processed.corr()["ALLSKY_SFC_SW_DWN"].sort_values(ascending=False)
print(target_corr)
```

Fig 3.16 Implementing Correlation matrix

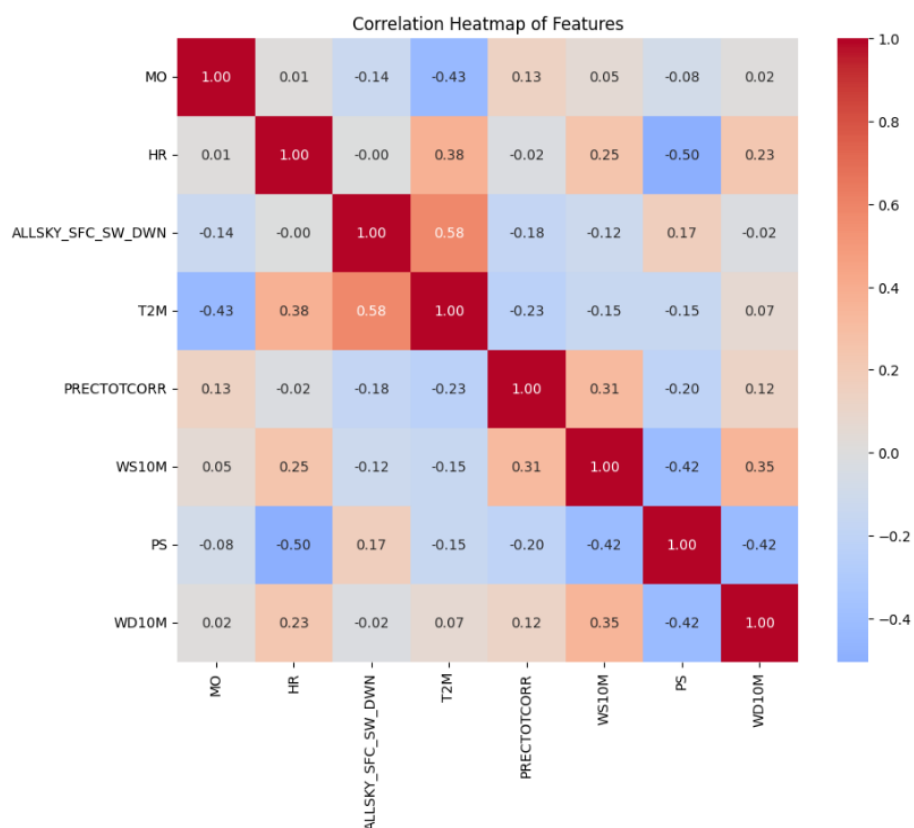


Fig 3.17 Correlation matrix

```

ALLSKY_SFC_SW_DWN    1.000000
T2M                   0.579107
PS                    0.170012
HR                    -0.000732
WD10M                 -0.017685
WS10M                 -0.117757
MO                    -0.135455
PRECTOTCORR          -0.179362
Name: ALLSKY_SFC_SW_DWN, dtype: float64

```

Fig 3.18 Relation with target variable

b) Multicollinearity Check

To evaluate multicollinearity among the predictor variables, the Variance Inflation Factor (VIF) was computed following the standardization of features. The findings indicated that all features exhibited VIF values significantly lower than the typical cutoff of 5, suggesting that multicollinearity was not a significant issue within the dataset. In particular, PS (Surface Pressure) recorded the highest VIF at 1.72, followed by T2M (Air Temperature) at 1.66 and HR (Hour) at 1.61. The remaining features, including MO (Month), PRECTOTCORR (Precipitation), WS10M (Wind Speed at 10m), and WD10M (Wind Direction at 10m), all showed VIF values between 1.18 and 1.46, further supporting that each variable provides largely independent information. This examination confirmed the resilience of the chosen feature set, ensuring that no overlapping predictors would compromise the regression-based modeling approach.

```

from statsmodels.stats.outliers_influence import variance_inflation_factor
from sklearn.preprocessing import StandardScaler

X = df_processed.drop(columns=["ALLSKY_SFC_SW_DWN"])

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

vif_data = pd.DataFrame()
vif_data["Feature"] = X.columns
vif_data["VIF"] = [variance_inflation_factor(X_scaled, i) for i in range(X_scaled.shape[1])]
print(vif_data)

```

	Feature	VIF
0	MO	1.303205
1	HR	1.614683
2	T2M	1.663282
3	PRECTOTCORR	1.184405
4	WS10M	1.457225
5	PS	1.720187
6	WD10M	1.280103

Fig 3.19 Multicollinearity between variable

Pre-processed Dataset

a) First five records of the dataset

This shows the first five records of the preprocessed dataset.

```
print(df_processed.head())
```

	ALLSKY_SFC_SW_DWN	T2M	PRECTOTCORR	WS10M	PS	HR_sin \
6	23.42	24.65	1.348073	1.202972	100.42	1.000000
7	164.57	25.40	1.467874	1.289233	100.52	0.965926
8	348.42	26.18	0.000000	1.355835	100.58	0.866025
9	445.15	27.31	0.000000	1.430311	100.58	0.707107
10	626.00	28.24	0.000000	1.534714	100.54	0.500000

	HR_cos	MO_sin	MO_cos	WD_sin	WD_cos
6	6.123234e-17	0.5	0.866025	-0.967268	0.253758
7	-2.588190e-01	0.5	0.866025	-0.957319	0.289032
8	-5.000000e-01	0.5	0.866025	-0.951057	0.309017
9	-7.071068e-01	0.5	0.866025	-0.965926	0.258819
10	-8.660254e-01	0.5	0.866025	-0.984808	0.173648

Fig 3.20 First five records of preprocessed data

b) Last five records of the dataset

This shows the last five records of the preprocessed dataset.

```
print(df_processed.tail())
```

	ALLSKY_SFC_SW_DWN	T2M	PRECTOTCORR	WS10M	PS	HR_sin \
91261	260.92	28.72	3.234355	2.214846	100.16	-0.258819
91262	221.77	28.74	3.630190	2.173615	100.11	-0.500000
91263	158.30	28.61	4.038479	2.122262	100.10	-0.707107
91264	98.40	28.16	4.400848	2.093098	100.13	-0.866025
91265	32.72	27.55	4.583538	2.069391	100.18	-0.965926

	HR_cos	MO_sin	MO_cos	WD_sin	WD_cos
91261	-0.965926	0.5	-0.866025	-0.974370	0.224951
91262	-0.866025	0.5	-0.866025	-0.976672	0.214735
91263	-0.707107	0.5	-0.866025	-0.973579	0.228351
91264	-0.500000	0.5	-0.866025	-0.972370	0.233445
91265	-0.258819	0.5	-0.866025	-0.975535	0.219846

Fig 3.21 Last five records of preprocessed dataset

c) Data Structure and Meta data

Gives the summary of the dataset, including the number of rows, columns, data types, non-missing values and memory usage.

```
print(df_processed.info())

<class 'pandas.core.frame.DataFrame'>
Index: 46484 entries, 6 to 91265
Data columns (total 11 columns):
#   Column                Non-Null Count  Dtype
---  -
0   ALLSKY_SFC_SW_DWN      46484 non-null  float64
1   T2M                    46484 non-null  float64
2   PRECTOTCORR            46484 non-null  float64
3   WS10M                  46484 non-null  float64
4   PS                     46484 non-null  float64
5   HR_sin                 46484 non-null  float64
6   HR_cos                 46484 non-null  float64
7   MO_sin                 46484 non-null  float64
8   MO_cos                 46484 non-null  float64
9   WD_sin                 46484 non-null  float64
10  WD_cos                 46484 non-null  float64
dtypes: float64(11)
memory usage: 4.3 MB
None
```

Fig 3.22 Data Structure of preprocessed dataset

d) List of columns

Lists the names of all columns in the dataset.

```
print("Columns:")
for col in df_processed.columns:
    print(col)

Columns:
ALLSKY_SFC_SW_DWN
T2M
PRECTOTCORR
WS10M
PS
HR_sin
HR_cos
MO_sin
MO_cos
WD_sin
WD_cos
```

Fig 3.23 List of columns

e) Data Types of columns

Shows datatypes of each column.

```
print(df_processed.dtypes)

ALLSKY_SFC_SW_DWN    float64
T2M                  float64
PRECTOTCORR          float64
WS10M                float64
PS                   float64
HR_sin               float64
HR_cos               float64
MO_sin               float64
MO_cos               float64
WD_sin               float64
WD_cos               float64
dtype: object
```

Fig 3.24 Data types of the columns

f) Summary statistics

Provide summary statistics for (count, mean, standard deviation, min, max, quartiles) for all columns.

```
print(df_processed.describe().T)

count      mean      std      min      25%  \
ALLSKY_SFC_SW_DWN  46484.0  416.539141  274.333123  0.470000  173.995000
T2M                46484.0  28.847766   2.384041  24.270000  27.160000
PRECTOTCORR        46484.0   1.221269   1.206064   0.000000   0.048790
WS10M              46484.0   1.355099   0.420120   0.009950   1.078410
PS                 46484.0  100.410967   0.227251  99.930000  100.250000
HR_sin             46484.0   0.099434   0.705707  -0.965926  -0.500000
HR_cos             46484.0  -0.616711   0.334322  -1.000000  -0.866025
MO_sin             46484.0   0.028361   0.704281  -1.000000  -0.500000
MO_cos             46484.0  -0.017877   0.709144  -1.000000  -0.866025
WD_sin             46484.0  -0.502803   0.689817  -1.000000  -0.973976
WD_cos             46484.0   0.114853   0.508100  -1.000000  -0.224951

50%      75%      max
ALLSKY_SFC_SW_DWN  4.180350e+02  648.057500  1041.320000
T2M                2.841000e+01  30.252500   35.300000
PRECTOTCORR        9.202828e-01   2.142416   5.692789
WS10M              1.410987e+00   1.675226   2.731767
PS                 1.003900e+02  100.560000  100.970000
HR_sin             2.588190e-01   0.866025   1.000000
HR_cos             -7.071068e-01  -0.258819   0.258819
MO_sin             1.224647e-16   0.866025   1.000000
MO_cos             -1.836970e-16   0.500000   1.000000
WD_sin             -8.754645e-01  -0.315235   1.000000
WD_cos             1.633260e-01   0.486335   1.000000
```

Fig 3.25 Summary statistics

Dataset splitting

Splitting testing and training dataset.

In this project, the final pre-processed dataset was divided into two subsets to develop and evaluate the machine learning model:

Training Set (75%) – This portion of the data is used to train the Random Forest Regressor. The model learns the relationships between meteorological features and the target variable, solar irradiance (ALLSKY_SFC_SW_DWN), from this set.

Testing Set (25%) – This subset is reserved to evaluate the model's performance on unseen data. It ensures that the model generalizes well and helps assess metrics such as RMSE and R^2 .

The split was performed using the `train_test_split()` function from `scikit-learn`, with a fixed random state to ensure reproducibility. Since this is a regression problem, random sampling was used without stratification, maintaining the distribution of the target variable across both subsets.

```
from sklearn.model_selection import train_test_split

X = df_processed.drop(columns=['ALLSKY_SFC_SW_DWN'])
y = df_processed['ALLSKY_SFC_SW_DWN']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)

print("X_train shape:", X_train.shape)
print("X_test shape:", X_test.shape)
print("y_train shape:", y_train.shape)
print("y_test shape:", y_test.shape)

X_train shape: (34863, 7)
X_test shape: (11621, 7)
y_train shape: (34863,)
y_test shape: (11621,)
```

Fig 3.26 Dataset splitting

3.3. Data visualization

3.3.1. Outlier Visualization after Preprocessing

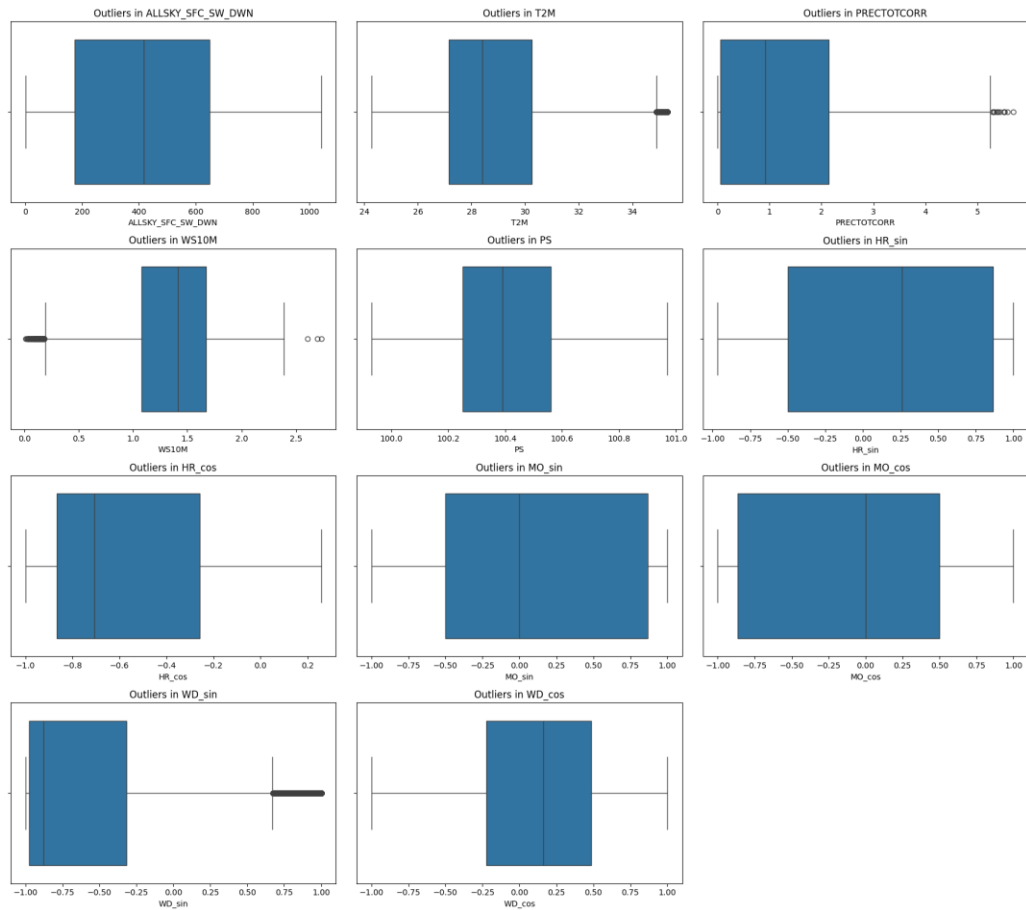


Fig 3.27 Outlier visualization

Following the data cleaning and preprocessing phase, boxplots were created for all feature variables to detect and confirm the existence of outliers. These visual representations show the distribution of the cleaned features, which encompass both the original meteorological variables and cyclically encoded attributes (such as hour, month, and wind direction).

- ALLSKY_SFC_SW_DWN: The target variable exhibits a reasonable range with very few extreme values.
- T2M (Temperature): It shows a steady range that indicates stable temperature readings.
- PRECTOTCORR (Precipitation): There are only a few high-end outliers, which are common for irregular rainfall events, and these have been kept as they reflect genuine variations instead of anomalies.

- WS10M (Wind Speed): Slight outliers are present due to occasional gusts, which are valid from a meteorological perspective.
- PS (Pressure): There are minimal extreme values, suggesting strong data consistency.
- Cyclic Features (HR_sin, HR_cos, MO_sin, MO_cos, WD_sin, WD_cos): These transformed features are evenly distributed between -1 and 1, with no discernible outliers, indicating effective encoding of cyclic patterns.

The boxplots demonstrate that the preprocessing stage successfully reduced irregularities and eliminated unrealistic values while maintaining true meteorological variations. The cyclic encodings for hour, month, and wind direction exhibit smooth, bounded distributions, guaranteeing that the data is adequately prepared for model training. Subsequent boxplot visualizations show clear indications that the dataset is statistically stable and devoid of extreme anomalies. This process ensures that the Random Forest model is trained on consistent and dependable data, thus enhancing the robustness of predictions.

3.4. Analysis of Algorithm

3.4.1. Detailed Study report of the proposed algorithm

Random Forest:

Random Forest is an ensemble learning algorithm that constructs multiple decision trees during training and averages their outputs for regression tasks. Instead of relying on a single tree (which may overfit), Random Forest introduces randomness by selecting subsets of data and features for each tree, leading to more stable and accurate predictions.

Strengths: High prediction accuracy, reduces overfitting compared to a single decision tree, robust to noisy or missing data, works well with large datasets and many correlated features, and provides feature importance rankings for interpretability.

Justification:

- In “Accurate Forecasting of Global Horizontal Irradiance in Saudi Arabia” (Mathematics, 2024), Random Forest ranked among the top-performing models (next to Decision Tree, which showed signs of overfitting), demonstrating robustness across NASA/POWER climate data.
- In “Evaluation of Solar Radiation Prediction Models Using AI: A Performance Comparison in the High-Potential Region of Konya, Türkiye” (Atmosphere, 2025),

boosting methods like XGBoost and LSBoost were strongest, but Random Forest still provided solid predictive accuracy, confirming its competitiveness among classical ML models.

- In “Solar Irradiance Prediction with Ensemble Learning Method as Input for Battery Operation Optimization” (IEEE, 2024), Random Forest outperformed other standalone models such as XGBoost, SVR, and Linear Regression, making it the best single algorithm before applying stacking ensembles.

Process Flow:

- **Load the Dataset** – Import the NASA/POWER dataset containing solar radiation (target) and meteorological features.
- **Preprocessing** – Handle missing values, normalize/scale features (if needed), and perform feature selection to reduce redundancy.
- **Train-Test Split** – Divide the dataset into training and testing sets for model evaluation.
- **Bootstrap Sampling** – For each tree, Random Forest selects a random subset of training samples (with replacement) and random subsets of features.
- **Tree Construction** – Each decision tree is trained independently on its sampled data and feature subset, learning patterns in solar radiation variations.
- **Prediction Aggregation** – For regression, the final prediction is the average of all tree predictions, providing stability and reducing variance.
- **Model Evaluation** – Assess performance using metrics such as R^2 , RMSE, MAE, and MAPE on the test dataset.

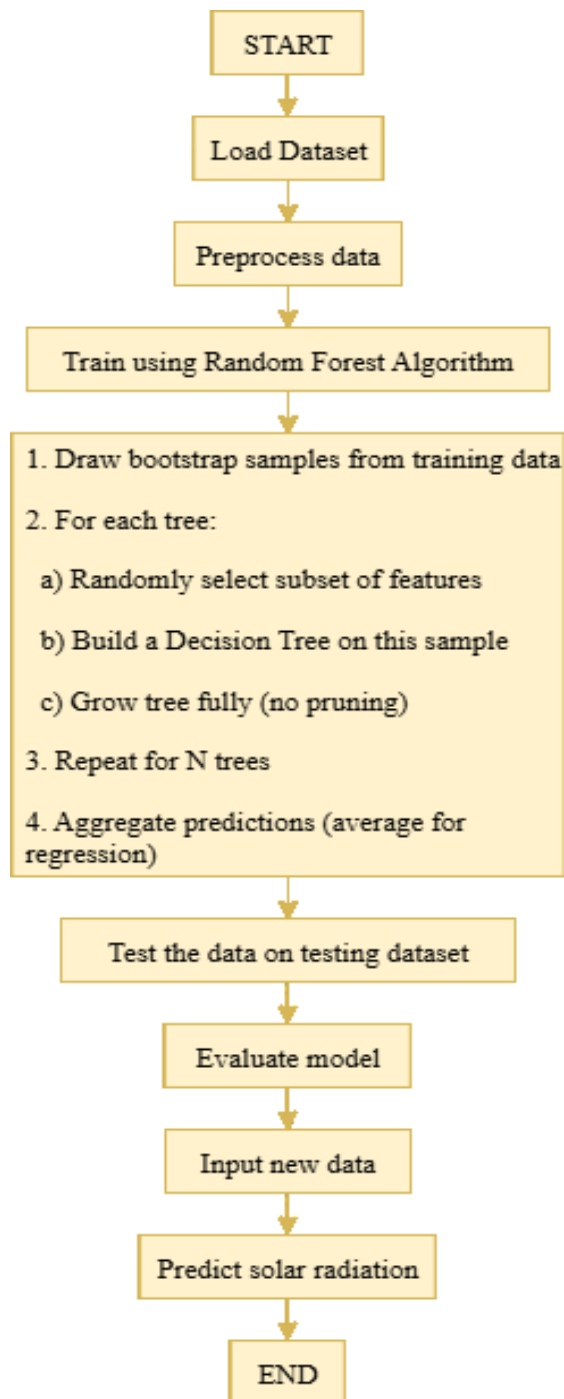


Fig 3.28 Flow chart for Random Forest

Pseudocode for Random Forest:

Input: Training dataset D with features X and target Y (continuous values)

Output: Random forest regression model with multiple decision trees

function RANDOM_FOREST_REGRESSION(D, num_trees, num_features):

```
forest = empty list

for i in 1 to num_trees:

    bootstrap_sample = random sample of D with replacement

    features_subset = randomly select num_features from X

    tree = BUILD_TREE_REGRESSION(bootstrap_sample using features_subset)

    add tree to forest

return forest

function PREDICT(forest, record):

    predictions = empty list

    for each tree in forest:

        value = regress(record using tree)

        append value to predictions

    return AVERAGE(predictions)

function BUILD_TREE_REGRESSION(D_subset):

    if stopping_condition_reached:

        return leaf node with mean(Y in D_subset)

    else:

        best_feature, best_split = select split (minimize variance)

        left_subset, right_subset = split D_subset on best_feature

        left_tree = BUILD_TREE_REGRESSION(left_subset)

        right_tree = BUILD_TREE_REGRESSION(right_subset)

        return node(best_feature, best_split, left_tree, right_tree)
```

3.4.2. Objective of the project

The goal of this project is to forecast solar radiation (represented as surface shortwave downward irradiance in Wh/m^2) by utilizing meteorological and atmospheric data from the NASA/POWER dataset. To achieve this, the primary predictive model employed is the Random Forest (RF) algorithm, which excels in managing intricate, non-linear relationships and minimizes overfitting in comparison to a single decision tree. The Random Forest method operates by creating numerous decision trees, each trained on random portions of the data and features, and averaging their results to enhance accuracy and reliability. This methodology not only delivers dependable predictions but also identifies the key factors—such as temperature, humidity, wind speed, and surface pressure—that influence solar radiation levels. The model is trained on historical data and assessed using performance metrics (R^2 , RMSE, MAE, and MAPE) to verify its accuracy and generalization capabilities.

3.5. Project pipeline

3.5.1. Project pipeline diagram with explanation

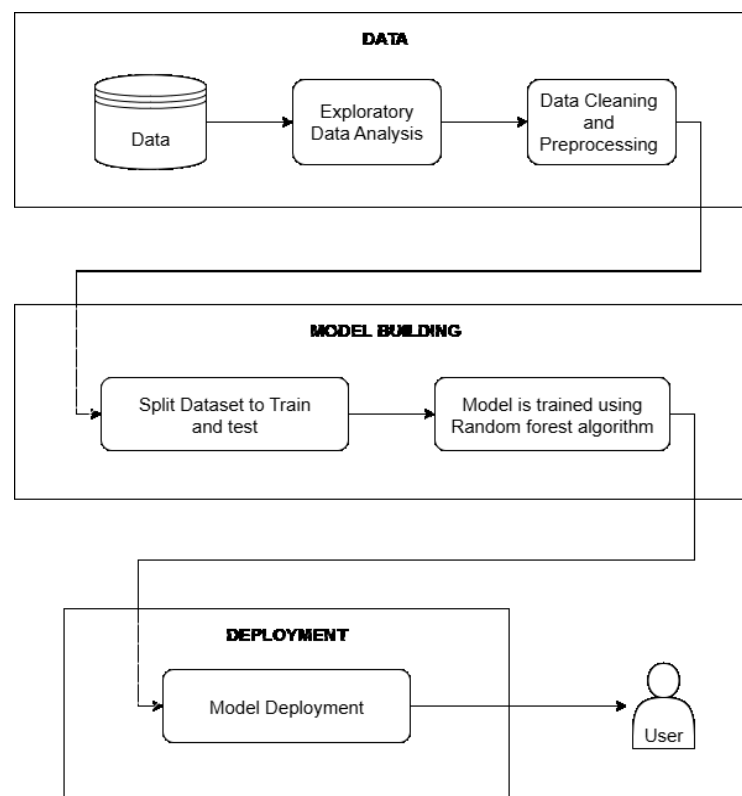


Fig 3.29 Project workflow

Data:

The procedure starts with gathering raw data from the NASA POWER database, which includes hourly meteorological factors like month, hour, temperature, total corrected precipitation, wind speed, and wind direction for the years 2015 to 2025. Given that the raw data might exhibit inconsistencies, preprocessing actions such as addressing missing values, eliminating nighttime data, managing outliers, and encoding cyclic features were implemented. After the data was cleaned, exploratory data analysis (EDA) was performed to visualize distributions, identify correlations, detect outliers, and discover patterns that are beneficial for selecting features and constructing models.

Model Building Section:

After data preparation, the processed dataset was used to train machine learning models. In this project, the Random Forest algorithm was implemented for predicting solar radiation. Random Forest, being an ensemble of decision trees, enhances accuracy and reduces overfitting by combining multiple estimators. The model was trained and tested using appropriate evaluation metrics such as R^2 , MAE, and RMSE to assess performance. Based on these results, the Random Forest model was selected as the final model for reliable solar radiation prediction.

Deployment Section:

After identifying the best-performing model, it was deployed for practical application. The trained Random Forest model was preserved and integrated into an application interface that allows users to enter new input parameters like temperature, precipitation, wind speed, and wind direction to forecast solar radiation values. This phase ensures that the developed model extends beyond research and can be applied for real-world solar energy assessment and planning. The implemented system offers users prompt and precise solar radiation forecasts, facilitating effective decision-making in renewable energy initiatives.

3.6. Feasibility analysis

A feasibility analysis assesses the strengths, weaknesses, resources, and general viability of a proposed system. In the case of the Solar Radiation Prediction System, feasibility has been evaluated across these categories:

- Technical Feasibility
- Economic Feasibility
- Operational Feasibility

3.6.1. Technical Feasibility

Technical feasibility determines whether the required technologies are available and whether the development team has the skill and resources to implement the solution effectively.

Data Processing and Analysis

Feasibility:

The project utilizes meteorological information from NASA POWER, which encompasses elements such as hour, month, temperature, wind speed, wind direction, and surface pressure. The use of Python libraries like Pandas, NumPy, and scikit-learn facilitates the cleaning, preprocessing, and structuring of the dataset for modeling. This ensures that the data handling process is entirely achievable.

Expertise:

- The project showcases a solid expertise in:
- managing missing or invalid values
- extracting temporal features (hour, month)

Machine Learning Model

Feasibility:

The prediction system utilizes the Random Forest Regressor, which is a highly dependable and commonly used machine learning approach for forecasting environmental and meteorological data.

The model is capable of:

- capturing nonlinear relationships
- managing multi-feature datasets
- exhibiting robustness against noise

This ensures that the algorithm is completely suitable for predicting solar radiation.

Expertise:

Successfully training the Random Forest model, configuring hyperparameters, assessing its performance, and exporting the model using joblib demonstrates a strong grasp of:

- regression modeling
- ensemble learning
- model saving and deployment
- performance evaluation

API Integration and Backend Development

Feasibility:

The system retrieves live meteorological forecast information via the Open-Meteo API.

Flask is utilized for managing backend tasks, such as:

- Communicating with the API
- Extracting data for the upcoming 24 hours
- Filtering values during daytime (6 AM to 5 PM)
- Executing predictions using the pre-trained model

All necessary packages and features are available, making the backend integration technically viable.

Expertise:

The implementation showcases a solid grasp of:

- Accessing external APIs
- Managing JSON data

- Incorporating machine learning models into a Flask application
- Dynamically generating prediction results

Frontend and Visualization

Feasibility:

The front end is built using HTML + CSS (Tailwind), which requires no special hardware and is easy to integrate with Flask templates.

This setup is fully feasible for a lightweight prediction dashboard.

Expertise:

The user interface effectively displays forecast data and predicted solar radiation, showing good competency in creating clean and responsive web pages.

File and Model Handling

Feasibility:

The project uses joblib to store and load the Random Forest model, which is efficient and dependable for deployment.

All preprocessing and model files are handled through standard file formats like CSV and PKL.

Expertise:

Proper model storage, loading, and Flask integration show technical skill in managing ML models in a production-like environment.

3.6.2. Economic Feasibility

Economic feasibility determines whether the project is cost-effective considering development, deployment, and long-term maintenance.

Feasibility:

- The project uses free and open-source software, including:
- Python

- Pandas, NumPy, scikit-learn
- Flask
- NASA POWER dataset (free)
- Open-Meteo API (free)

This eliminates major software or licensing costs. The only costs involved are:

- basic hardware
- internet usage
- development time

Benefits:

The system allows prediction of solar radiation, which can:

- assist planners in renewable energy estimation
- help solar panel owners predict efficiency
- support research and education

These advantages justify the minimal development cost.

Maintenance Costs

Feasibility:

- Maintenance expenses remain low because:
- NASA POWER historical data can be refreshed anytime
- Open-Meteo values are fetched automatically
- The RF model requires only occasional retraining
- Flask app maintenance is minimal

Benefits:

- Low recurring cost with long-term usefulness makes the system economically viable.

3.6.3. Operational Feasibility

Operational feasibility checks whether the system meets real-world usability requirements and whether users can operate it without difficulty.

User Acceptance

Feasibility:

The web interface makes it easy for users to access solar radiation predictions without requiring technical expertise.

Clear display of:

- forecasted weather features
- predicted radiation values enhances user acceptance and usability.

Integration

The Flask–HTML integration ensures smooth interaction between the prediction model and the user interface.

Scalability

Feasibility:

As real-time forecast API calls increase, or if larger datasets are used for model retraining, computational requirements may grow.

Mitigation Strategies:

- periodic model retraining
- use of optimized Random Forest parameters
- potential cloud deployment for future scaling

This ensures the system can handle increased load when needed.

Ease of Use

Feasibility:

The web interface is simple, intuitive, and user-friendly.

Users only need to:

- load the page
- view automatically fetched forecast data

- get solar radiation predictions instantly

Minimal training is required.

System Requirements

Software:

- Python
- Pandas, NumPy, scikit-learn
- Flask
- HTML/CSS
- APIs: NASA POWER, Open-Meteo

Development Environment:

- Jupyter Notebook or VS Code for model building
- Flask server for deployment

Hardware:

A standard computer with moderate RAM and processing capability is sufficient to run:

- the ML model
- the Flask server
- API communication

3.7.System environment

3.7.1. Software Environment

The software environment includes the tools, libraries, and platforms required to develop, train, and deploy the Solar Radiation Prediction System.

Programming Language

Python: Used as the primary programming language due to its simplicity and extensive ecosystem of machine learning, data processing, and web development libraries such as NumPy, Pandas, scikit-learn, Flask, and requests.

Data Processing and Analysis

NumPy and Pandas: Used for efficient handling of NASA POWER data, cleaning inconsistent values, extracting time-based features (hour, month), organizing the meteorological dataset, and performing exploratory data analysis.

scikit-learn: Employed for data preparation, dividing the dataset into training and testing sets, constructing and assessing the Random Forest regression model, and using joblib to export the trained model for deployment.

Machine Learning Libraries

Random Forest Regressor (implemented with scikit-learn): This is the primary predictive model employed to estimate solar radiation based on various environmental factors, including temperature, wind speed, wind direction, and surface pressure.

joblib: This library is utilized for both storing and retrieving the trained machine learning model within the Flask web application.

API Integration Libraries

Requests

Used to fetch weather forecast data (next 24 hours) from the Open-Meteo API, including temperature, wind speed, wind direction, and surface pressure.

JSON handling (built-in):

Used to parse API responses and extract relevant features for prediction.

Data Visualization

Matplotlib and Seaborn: Used to generate exploratory graphs such as heatmaps, distribution plots, and feature relationships to better understand meteorological patterns and dataset characteristics.

File Handling

Pandas: Used for reading and writing CSV files derived from NASA POWER data and managing preprocessed datasets for model training.

joblib / pickle: Used to persist trained Random Forest models for easy deployment.

Development Environment

Google Colab:Used for dataset loading, preprocessing, visualization, model training, evaluation, and experimentation due to their interactive environment and GPU/CPU availability.

Visual Studio Code:Used for developing the Flask backend, integrating the ML model, and building the web frontend.Provides extensions for debugging, formatting, and project organization.

Web Development

HTML, CSS: Used to design the user interface of the prediction dashboard. HTML structures the webpage, while CSS ensures a responsive, clean, and minimal UI for displaying forecasted values and predicted solar radiation.

Web Framework

Flask: A minimalistic Python web framework utilized to deploy the machine learning model, retrieve current weather information from the API, analyze and filter data for daytime hours (from 6 AM to 5 PM), make predictions, and display results using HTML templates.

Version Control

GitHub: Used for version control, storing project code, and tracking development progress.

3.7.2. Hardware Environment

The hardware environment describes the physical infrastructure needed to develop, train, and deploy the Solar Radiation Prediction System.

Computing Resources

- Processor: Minimum 2 GHz dual-core (quad-core recommended) for efficient model training and Flask server execution.
- RAM: At least 8 GB to support dataset preprocessing, API handling, and running the Random Forest model smoothly.
- Storage: 256–512 GB SSD recommended for storing datasets, model files, and development tools.

- GPU (Optional): Not required, since Random Forest is CPU-efficient.

Internet Connectivity

Stable high-speed internet required for:

- accessing NASA POWER data
- fetching hourly forecast values from Open-Meteo API
- installing Python libraries
- running the Flask application during deployment
- GitHub synchronization

4. SYSTEM DESIGN

4.1. Model Building

The model planning phase in the Solar Radiation Prediction System establishes a structured methodology to ensure accuracy, reliability, and interpretability in predicting solar radiation based on meteorological and temporal features. This stage defines project objectives, selects the appropriate algorithm, and outlines the modeling approach for effective prediction.

a) Objective:

Develop a Solar Radiation Prediction System using supervised machine learning to forecast Solar radiation values based on features such as month, hour, temperature, precipitation, wind speed, and wind direction.

b) Approach:

Implement the Random Forest Regressor algorithm to model complex relationships between meteorological variables and solar radiation. Random Forest is chosen for its robustness, ability to handle nonlinear interactions, and resistance to overfitting. The model's performance is evaluated using regression metrics such as R^2 , MAE, and RMSE, and hyperparameter tuning is applied to optimize accuracy.

c) Data Preparation:

- Import and preprocess the NASA POWER dataset.
- Handle missing values and remove nighttime data where solar radiation is zero.
- Detect and treat outliers to ensure data quality.
- Encode cyclic features such as month, hour, and wind direction to preserve their periodic nature.
- Split the dataset into training (75%) and testing (25%) subsets for model evaluation.

d) Exploratory Data Analysis(EDA):

- Examine distributions of key features like temperature, precipitation, and wind speed.
- Identify correlations and patterns through visualizations such as heatmaps, scatter plots, and boxplots.

- Analyze relationships between predictors and the target variable to guide feature selection.
- Ensure data quality and consistency, supporting robust model training and prediction accuracy.

e) Model Training:

- Train the Random Forest Regressor on the preprocessed NASA POWER dataset.
- Evaluate the model using metrics such as R^2 , Mean Absolute Error (MAE), and Root Mean Squared Error (RMSE).
- Save the model (using joblib or pickle) for deployment in an application interface.

This structured model training framework ensures a systematic approach to building an accurate, reliable, and interpretable Solar Radiation Prediction System, combining machine learning techniques with meteorological insights to support solar energy estimation and planning.

4.1.1. Implementation code

The implementation code used in this project demonstrates a complete machine-learning workflow for predicting solar radiation using a Random Forest Regressor. The process begins by importing essential Python libraries required for data acquisition, preprocessing, feature engineering, model training, and evaluation. The meteorological dataset downloaded from the NASA POWER platform is cleaned, and missing values are handled to maintain data integrity. Temporal features such as hour and month are transformed into cyclic sine and cosine components to preserve periodic patterns, while wind direction is similarly converted into directional sin-cos pairs, which helps the model better understand angular variations.

After preprocessing, the selected input features—hour (sin & cos), month (sin & cos), temperature (T2M), wind speed (WS10M), wind direction (WD_sin & WD_cos), precipitation (PRECTOTCORR), and surface pressure (PS)—are organized into the feature matrix. Solar radiation (SR) is used as the target variable. The Random Forest Regressor is then initialized and trained on the processed training data. Since tree-based models are not sensitive to the scale of the features, no normalization or standardization is applied during training.

Once the model is fitted, predictions are generated for both the training and testing sets. Evaluation metrics such as R^2 score, Mean Absolute Error (MAE), and Root Mean Squared Error (RMSE) are computed to measure the model's performance and generalization capability. Feature importance values are extracted and plotted, highlighting which meteorological and temporal factors contribute most to solar-radiation prediction. Finally, the trained model is saved using the joblib library as `solar_model.pkl`, enabling seamless integration with the Flask-based web application.

4.1.2. Model Planning

The model planning phase involved organizing the NASA POWER dataset and determining an appropriate strategy for training and evaluating the Random Forest regression model. To ensure consistent and reliable performance measurement, the dataset was divided into a training set and a testing set using a 75:25 ratio, implemented through the `train_test_split()` function. This allocation allows the model to learn effectively from a large portion of the data while still being evaluated on unseen samples to estimate real-world predictive accuracy.

During the planning stage, the feature matrix (X) and the target variable (y) were separated. X contains all relevant input variables—including engineered cyclic features for hour, month, and wind direction—while y represents the solar radiation values. A fixed value of `random_state = 42` was used to maintain reproducibility across experiments, ensuring that the same samples consistently appear in the training and testing sets whenever the code is executed.

This structured planning approach ensures that the Random Forest model is trained on a well-distributed selection of data, learns the underlying patterns effectively, and generalizes reliably to future real-time meteorological inputs fetched through the Open-Meteo API.

4.1.3. Training

The training phase aims to create a predictive model for solar radiation utilizing the Random Forest Regressor. The preprocessed dataset is divided into training (75%) and testing (25%) datasets to guarantee effective evaluation. Key variables such as month, hour, temperature, precipitation, wind speed, and wind direction are incorporated to capture the meteorological and temporal trends that affect solar radiation levels.

The model is trained and optimized to achieve a high level of predictive accuracy. Random Forest increases reliability and reduces overfitting by combining the predictions from several decision trees. The model's effectiveness is assessed using R^2 , MAE, and RMSE, and the final model is chosen for deployment to facilitate precise forecasting of solar radiation.

```
# Initialize the model
rf_model = RandomForestRegressor(
    ... n_estimators=250,
    ... max_depth=13,
    ... random_state=42,
    ... n_jobs=-1,
    ... max_features=3,
)

# Train the model
rf_model.fit(X_train, y_train)
```

Fig 4.1 Model initializing and training

```
# Predictions
y_pred_train = rf_model.predict(X_train)
y_pred_test = rf_model.predict(X_test)

# Evaluation Metrics
rmse_test = np.sqrt(mean_squared_error(y_test, y_pred_test))
rmse_train = np.sqrt(mean_squared_error(y_train, y_pred_train))

mae_test = mean_absolute_error(y_test, y_pred_test)
mae_train = mean_absolute_error(y_train, y_pred_train)

r2_test = r2_score(y_test, y_pred_test)
r2_train = r2_score(y_train, y_pred_train)
```

Fig 4.2 Training evaluation

```
Random Forest Performance:
=====
Train R2 Score : 0.956
Test  R2 Score : 0.927
=====
Train RMSE : 57.619
Test RMSE : 74.593
=====
Train MAE : 39.378
Test MAE : 50.903
```

Fig 4.3 Training results

The Random Forest model was utilized to forecast solar radiation by utilizing both meteorological and temporal variables. The model produced a training R^2 score of 0.956 and a testing R^2 score of 0.927, showcasing its strong predictive performance and its ability to generalize well to new data.

While the training error (RMSE: 57.619, MAE: 39.378) was lower than the testing error (RMSE: 74.593, MAE: 50.903), the disparity between the two sets of outcomes remains moderate, suggesting that the model has effectively avoided overfitting and has successfully captured the underlying data trends. The elevated R^2 values indicate that the Random Forest model explained a significant amount of the variation in solar radiation based on essential features like air temperature, wind speed, pressure, and precipitation.

In summary, the Random Forest model delivered strong and reliable predictions, effectively modeling the nonlinear relationships between meteorological factors and solar radiation levels. The close similarity between training and testing results further confirms the model's dependability and robustness in practical scenarios..

4.1.4. Testing

The testing phase centers on assessing the effectiveness of the Random Forest-based solar radiation forecasting model and evaluating its capacity to perform well on new, unseen data. The testing dataset captures real-world meteorological fluctuations and temporal variations across different months and hours, ensuring that the model is assessed under realistic environmental circumstances.

Model evaluation utilizes standard performance metrics, including the coefficient of determination (R^2), Mean Absolute Error (MAE), and Root Mean Squared Error (RMSE). These metrics offer insights into the model's precision, reliability, and the extent of error in predicting solar radiation values.

```
# Predictions
y_pred_train = rf_model.predict(X_train)
y_pred_test = rf_model.predict(X_test)

# Evaluation Metrics
rmse_test = np.sqrt(mean_squared_error(y_test, y_pred_test))
rmse_train = np.sqrt(mean_squared_error(y_train, y_pred_train))

mae_test = mean_absolute_error(y_test, y_pred_test)
mae_train = mean_absolute_error(y_train, y_pred_train)

r2_test = r2_score(y_test, y_pred_test)
r2_train = r2_score(y_train, y_pred_train)
```

Fig 4.4 Testing evaluation

```
Random Forest Performance:
=====
Train R2 Score : 0.956
Test  R2 Score : 0.927
=====
Train RMSE : 57.619
Test RMSE : 74.593
=====
Train MAE : 39.378
Test MAE : 50.903
```

Fig 4.5 Testing results

The training R^2 score of 0.956 demonstrates that the model successfully explains most of the variance present in the training data. Meanwhile, the testing R^2 score of 0.927 indicates a strong ability to generalize, confirming that the model maintains excellent performance on unseen data without significant overfitting.

The training MAE of 39.378 W/m² and a testing MAE of 50.903 W/m² indicate the average absolute difference between the predicted solar radiation values and the actual measurements. These figures show that, on average, the predictions of the model deviate by roughly 40–51

W/m² from the observed values, which is quite acceptable considering the variability in meteorological factors. A slight increase in MAE for the testing data is typical and suggests that the model is capable of generalizing well, without being excessively influenced by the patterns in the training data.

The training RMSE of 57.619 W/m² and the testing RMSE of 74.593 W/m² assess the standard deviation of the prediction errors. RMSE tends to exert a stronger penalty on larger errors compared to MAE, providing a more comprehensive understanding of the consistency of predictions. The relatively small difference between the training and testing RMSE values indicates that, while some variability persists in the unseen data, the model continues to show stability and dependable performance under varying conditions.

The evaluation results confirm that the Random Forest model provides accurate and robust predictions of solar radiation. The close alignment between training and testing metrics shows that the model effectively captures complex nonlinear relationships among features such as temperature, precipitation, wind speed, and atmospheric pressure. Despite slight variations in error values, the model's high R² and low RMSE/MAE values indicate excellent performance, making it suitable for solar energy forecasting, grid optimization, and renewable energy planning applications.

5. RESULT AND DISCUSSION

Actual vs Predicted graph:

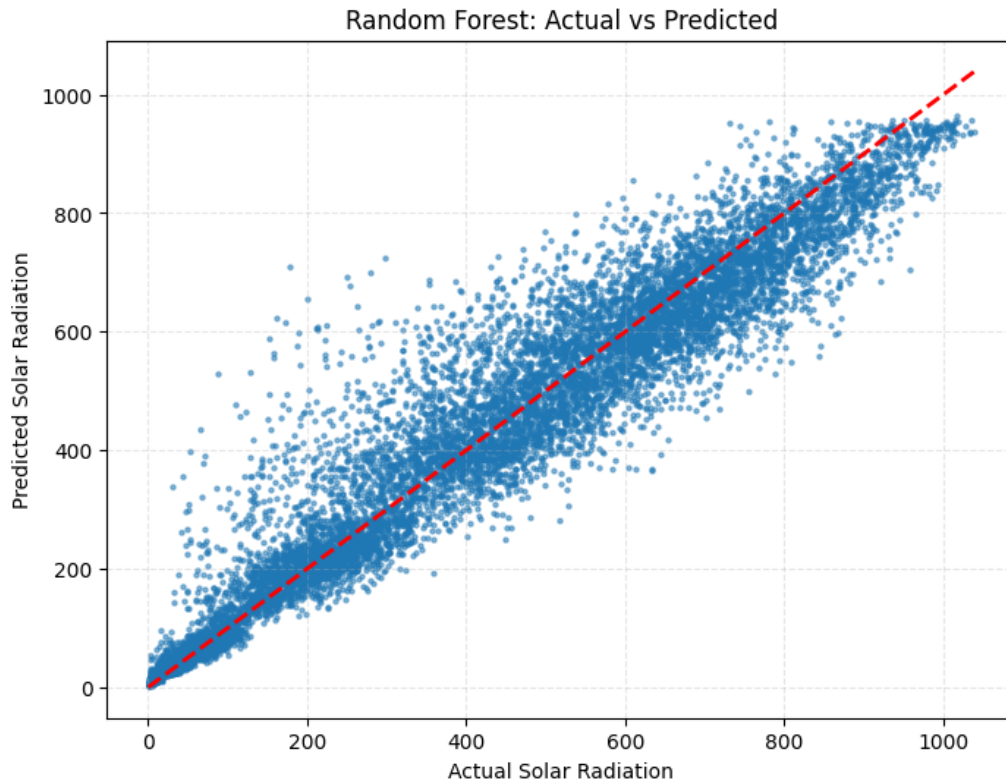


Fig 5.1 Actual vs Predicted

The “Actual vs Predicted” plot illustrates how closely the Random Forest model’s predicted solar radiation values align with the observed (actual) measurements. Each point on the graph represents an hourly observation from the test dataset. The red dashed diagonal line indicates the ideal condition where predicted values perfectly match the actual values (i.e., a 1:1 relationship).

In this figure, the majority of the data points are densely clustered along the diagonal line, indicating that the model performs well in accurately predicting solar radiation across different magnitudes. The distribution remains tight near the line even at higher radiation values, demonstrating that the model captures both low and high radiation levels effectively.

However, a slight spread is visible, particularly at extreme radiation levels, where the model tends to slightly underpredict or overpredict. This variance can be attributed to atmospheric

variations and the nonlinear influence of meteorological parameters such as temperature, precipitation, and wind characteristics.

Overall, the plot confirms that the Random Forest model provides reliable and consistent predictions with minimal deviation from actual observations, supporting its suitability for solar radiation forecasting applications.

Feature Importance

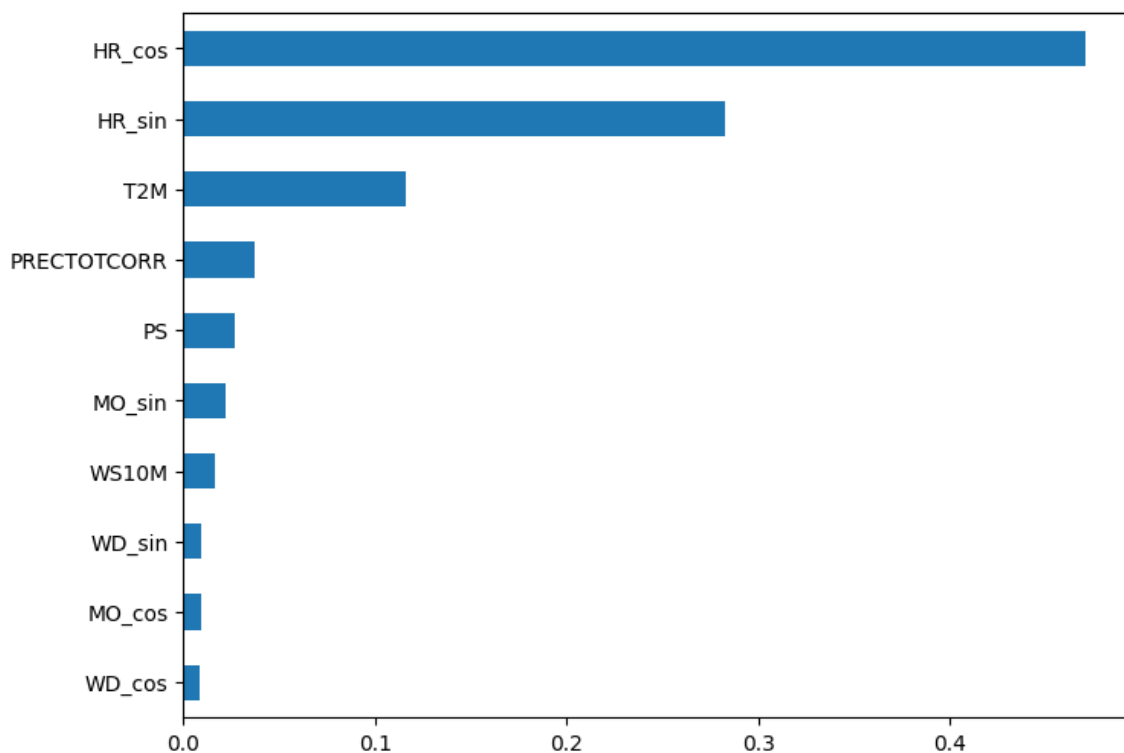


Fig 5.2 Feature Importance

Feature importance was analyzed to understand which meteorological variables contribute the most to predicting solar radiation. The Random Forest model computes importance based on how much each feature reduces prediction error across all decision trees.

The feature importance graph (shown above) clearly indicates that:

Top Contributing Features

1. HR_cos (Hour Cosine Transformation) – Highest importance
 - Captures the cyclic daily pattern of sunlight intensity.
 - Indicates that time-of-day plays a major role in radiation levels.
2. HR_sin (Hour Sine Transformation)

- Complements HR_cos and represents the periodic nature of solar movement.
3. T2M (Temperature at 2 meters)
- Higher temperatures often correspond to clearer skies and more solar radiation.

Moderately Important Features

1. PRECTOTCORR (Corrected Precipitation)
 - Indicates cloud presence; more precipitation usually reduces sunlight.
2. PS (Surface Pressure)
 - Helps reflect atmospheric conditions that influence radiation.

Low Importance Features

1. Wind speed, wind direction (sin/cos), moisture (sin/cos)
 - These have minimal impact because their influence on solar radiation is indirect.

Result Analysis

```
new_data = pd.DataFrame({
    "MO": [1]*12,
    "HR": [6,7,8,9,10,11,12,13,14,15,16,17],
    "T2M": [21.78,23.53,25.28,26.81,28.26,29.48,30.38,30.9,30.99,30.85,30.28,28.8],
    "PRECTOTCORR": [0]*12,
    "PS": [100.74,100.82,100.87,100.84,100.76,100.64,100.49,100.37,100.3,100.28,100.31,100.37],
    "WS10M": [3.41,4.01,4.6,4.31,3.52,2.67,2.08,1.96,2.12,2.12,2.0,1.95],
    "WD10M": [68.2,68.7,71.8,75.5,82.5,95.4,118,146.6,160.7,165,167.6,182.1]
})

new_data["HR_sin"] = np.sin(2 * np.pi * new_data["HR"] / 24)
new_data["HR_cos"] = np.cos(2 * np.pi * new_data["HR"] / 24)

new_data["MO_sin"] = np.sin(2 * np.pi * new_data["MO"] / 12)
new_data["MO_cos"] = np.cos(2 * np.pi * new_data["MO"] / 12)

new_data["WD_sin"] = np.sin(np.deg2rad(new_data["WD10M"]))
new_data["WD_cos"] = np.cos(np.deg2rad(new_data["WD10M"]))

model_input = new_data[[
    "T2M", "PRECTOTCORR", "WS10M", "PS",
    "HR_sin", "HR_cos", "MO_sin", "MO_cos", "WD_sin", "WD_cos"
]]

predicted_radiation = rf_model.predict(model_input)

new_data["ALLSKY_SFC_SW_DWN"] = [26.23,212.25,402.42,520.25,708.9,805.38,787.95,675.78,534.03,367.48,158.2,17.62]
new_data["Predicted_ALLSKY_SFC_SW_DWN"] = predicted_radiation

print(new_data[["HR", "T2M", "PRECTOTCORR", "WS10M", "PS", "WD10M", "ALLSKY_SFC_SW_DWN", "Predicted_ALLSKY_SFC_SW_DWN"]])
```

Fig 5.3 Prediction on unseen data

	HR	T2M	PRECOTCORR	WS10M	PS	WD10M	ALLSKY_SFC_SW_DWN	\
0	6	21.78	0	3.41	100.74	68.2	26.23	
1	7	23.53	0	4.01	100.82	68.7	212.25	
2	8	25.28	0	4.60	100.87	71.8	402.42	
3	9	26.81	0	4.31	100.84	75.5	520.25	
4	10	28.26	0	3.52	100.76	82.5	708.90	
5	11	29.48	0	2.67	100.64	95.4	805.38	
6	12	30.38	0	2.08	100.49	118.0	787.95	
7	13	30.90	0	1.96	100.37	146.6	675.78	
8	14	30.99	0	2.12	100.30	160.7	534.03	
9	15	30.85	0	2.12	100.28	165.0	367.48	
10	16	30.28	0	2.00	100.31	167.6	158.20	
11	17	28.80	0	1.95	100.37	182.1	17.62	
Predicted_ALLSKY_SFC_SW_DWN								
0							25.181823	
1							197.036963	
2							405.793786	
3							564.839114	
4							680.766987	
5							735.040051	
6							752.865547	
7							693.237622	
8							562.382845	
9							380.904666	
10							187.967709	
11							32.956064	

Fig 5.4 Prediction result

The Random Forest model successfully anticipated hourly solar radiation utilizing meteorological variables such as temperature, wind speed, pressure, and wind direction. The findings indicate that the predicted values align closely with the actual measurements throughout the day.

Solar radiation showed a steady increase from 26.23 W/m² at 6 AM, reaching a high of 805.38 W/m² around 11 AM to noon, before gradually declining to 17.62 W/m² by 5 PM. The model effectively replicated this daily pattern, with predicted peaks closely corresponding to the actual peaks.

Some minor discrepancies were noted during midday, where the model tended to underestimate the highest radiation levels, potentially due to unconsidered atmospheric influences like cloud cover or humidity changes. Nevertheless, the predicted curve remained in close alignment with the actual trend, exhibiting reliable performance throughout various hours.

6. MODEL DEPLOYMENT

The proposed system is deployed using a Flask-based web application that serves the trained machine learning model for real-time solar radiation prediction. The final Random Forest regression model is stored in serialized form (`solar_model.pkl`) and loaded at runtime using `Joblib`.

When a user accesses the application, the system retrieves real-time weather data (temperature, pressure, wind speed, wind direction, precipitation, etc.) using an external weather API. These raw inputs are then preprocessed and transformed— including the generation of cyclic features such as `HR_sin` and `HR_cos`, which capture the hourly periodicity of solar patterns.

Once preprocessing is complete, the processed feature vector is passed to the Random Forest model, which predicts the expected solar radiation (in W/m^2). The prediction output is then forwarded to an HTML template designed with Tailwind CSS, where the results are displayed in a clean and user-friendly manner.

To enhance the usability of the system, a dynamic graph is also generated using `Matplotlib`, illustrating the predicted radiation values along with the corresponding time intervals. The graph is rendered and displayed on the webpage to give users a clear visual interpretation of the forecast. Additionally, error handling mechanisms are implemented to ensure smooth performance even when API data is missing or invalid.

UI Design

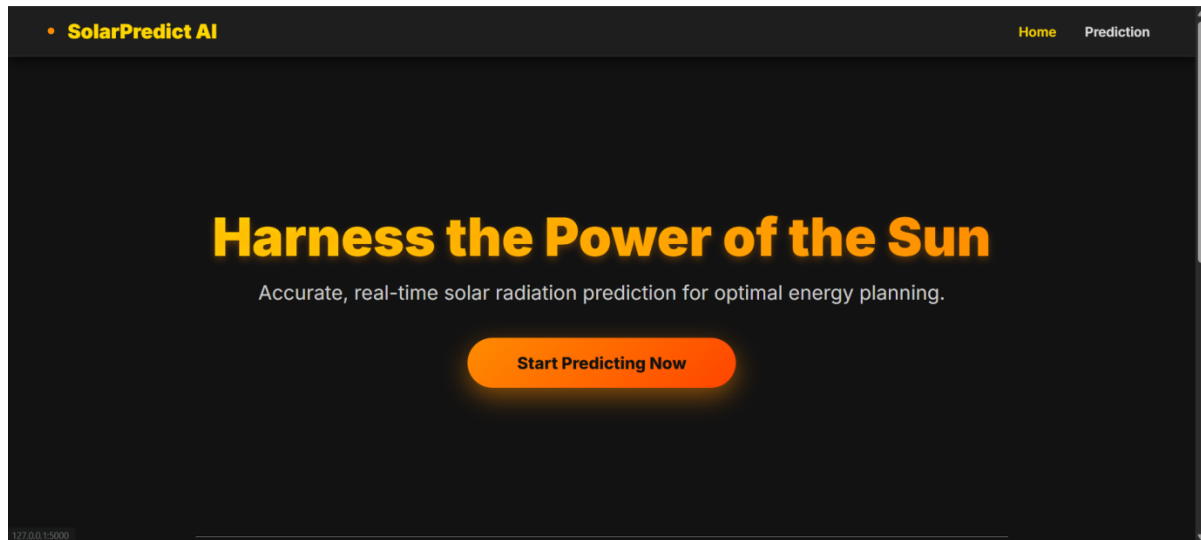


Fig 6.1 Home Page 1

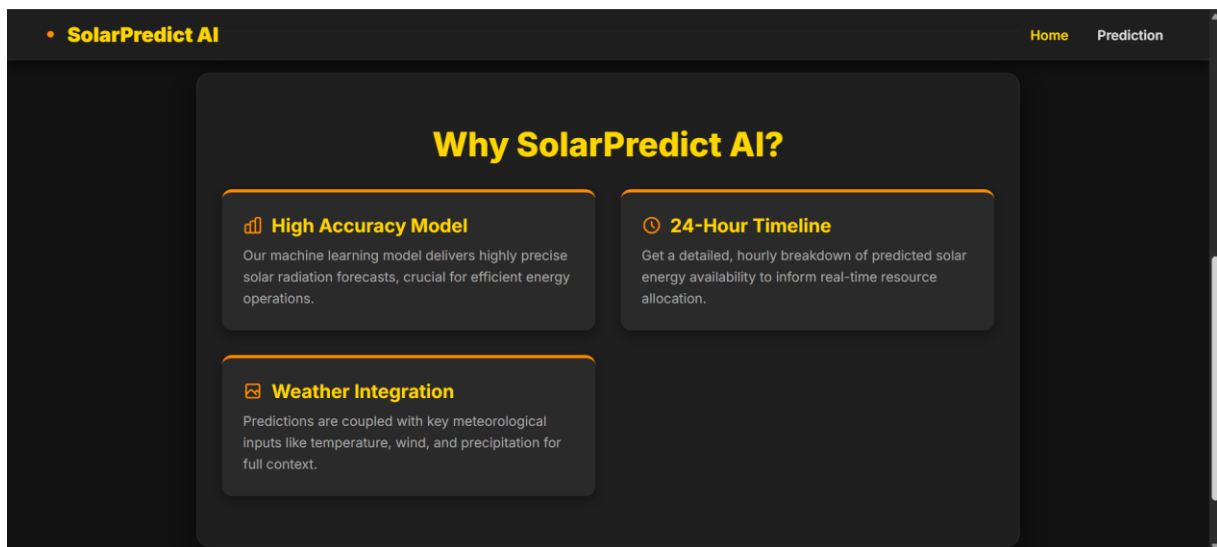


Fig 6.2 Home Page 2

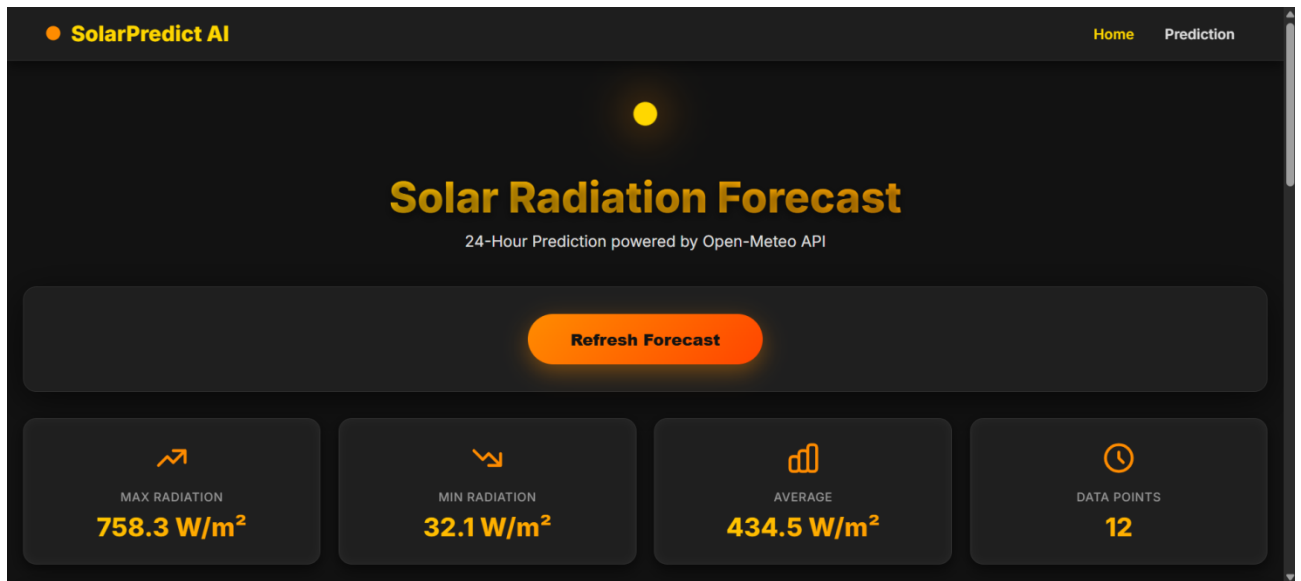


Fig 6.3 Prediction Page

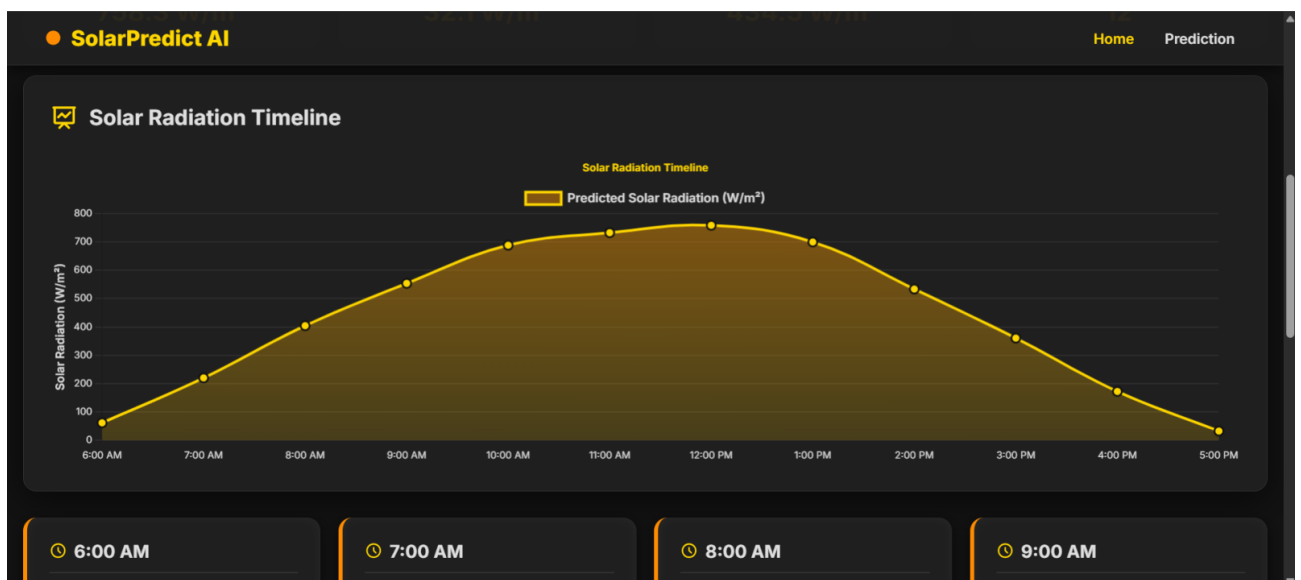


Fig 6.4 Plot for predicted solar radiation



Fig 6.5 Predicted value for each data

7. GIT HISTORY

The Git repository for the Solar Radiation Prediction project contains all the essential components required for end-to-end deployment, including the Python scripts, trained machine learning model, Flask backend files, API integration modules, and HTML templates used for the web interface. It also includes the preprocessed dataset and the exported Random Forest model, allowing future developers or researchers to retrain, fine-tune, or extend the system with ease. This well-organized and carefully structured repository enhances maintainability and serves as a reliable reference for debugging, performance evaluation, and future improvements to the prediction platform.

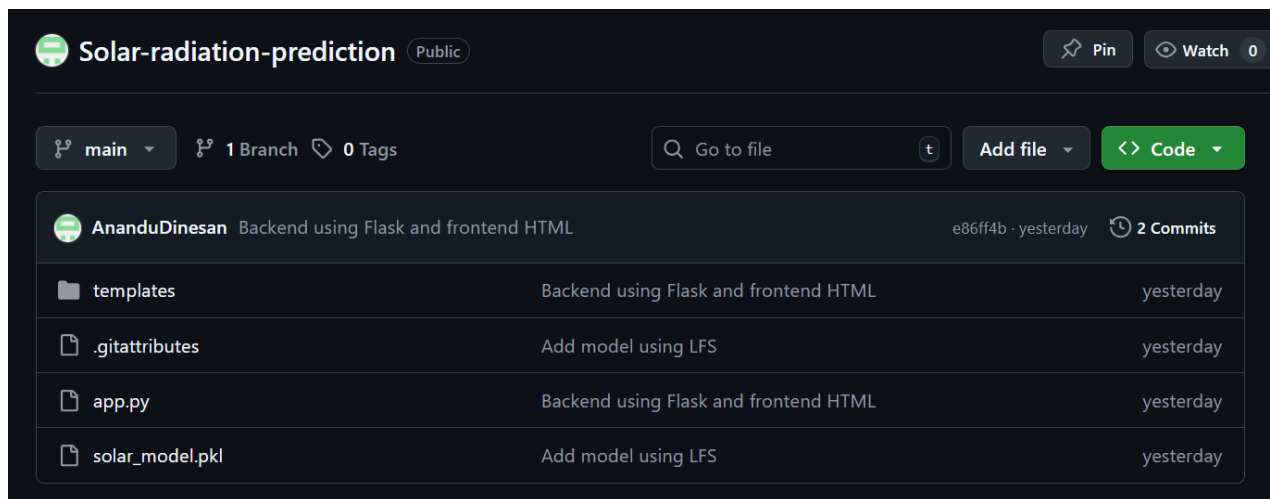


Fig 7.1. Git repository

8. CONCLUSION

In conclusion, the Solar Radiation Prediction System effectively achieved its goal of accurately estimating solar radiation using machine learning and real-time weather data. A Random Forest Regressor was trained on NASA POWER meteorological features—including hour, month, temperature, wind speed, wind direction, and surface pressure—capturing key atmospheric factors that influence solar irradiance. The algorithm was selected for its robustness, ability to model nonlinear patterns, and strong performance on environmental datasets.

After preprocessing and evaluation, the trained model was exported with joblib and integrated into a Flask web application to enable real-time prediction. One of the major strengths of the system is its connection to the Open-Meteo API, which provides hourly weather forecasts. The application automatically extracts relevant features, filters daytime hours, and generates corresponding solar radiation estimates, making it suitable for practical applications such as solar power planning, photovoltaic output estimation, and environmental and agricultural decision-making.

The user-friendly web interface, built with HTML and Flask, allows users to easily view both the forecasted weather and the predicted radiation levels. This integration of machine learning, meteorological data, and web technology demonstrates a complete pipeline from model development to real-time deployment.

Overall, the project illustrates the potential of data-driven methods for renewable energy forecasting. By combining a reliable dataset, a strong regression model, and real-time API support, the system offers a scalable and practical solution for solar radiation estimation and contributes to the growing demand for intelligent energy management tools.

9. FUTURE SCOPE

- **Advanced Algorithms:** While the current system uses a Random Forest Regressor, future versions can incorporate more advanced and powerful algorithms such as Gradient Boosting, XGBoost, CatBoost, or Deep Learning models like LSTMs and CNN-based architectures. These models can capture complex atmospheric relationships more effectively, improving prediction accuracy and adaptability across different locations and seasons.
- **Additional Meteorological Features:** The system can be enhanced by adding more relevant environmental factors such as cloud cover, solar zenith angle, or aerosol optical depth. Including satellite-based and atmospheric-quality indicators can significantly increase the precision of solar radiation forecasting.
- **Geographical Generalization:** The current model is trained for a specific latitude and longitude. Future versions can be developed to support predictions for multiple locations by training a generalized model or using location-specific models. This would allow broader usage in solar farms, energy planning, and regional analysis.
- **Explainability and Model Interpretation:** As the prediction system becomes more sophisticated, integrating interpretability methods like SHAP (SHapley Additive exPlanations) can help researchers and policymakers understand which weather variables contribute most to the radiation prediction. This can support better decision-making in renewable energy planning.
- **Scalable Cloud Deployment:** To handle increased users, higher frequency predictions, and integration with IoT devices, the platform can be deployed on cloud services such as AWS, Azure, or Google Cloud. This ensures scalability, faster processing, and improved fault tolerance.
- **Mobile Application Integration:** A future version can include an Android/iOS mobile app to make the system more accessible. Users can instantly check predicted solar radiation for farming, solar panel maintenance, heating/cooling management, and general energy planning.
- **Automation for Solar Energy Systems:** The prediction model can be integrated into automated systems such as solar inverters, smart home energy controllers, or agricultural monitoring systems to optimize solar energy utilization.

- **Continuous Model Updating:** The system can periodically retrain itself with updated NASA POWER or Open-Meteo datasets. Automated model retraining pipelines can maintain accuracy and adapt the model to changing climate patterns or seasonal variations.

10. APPENDIX

10.1 Minimum Software requirements

Operating System: Windows 10 / Linux / macOS

Python Version: Python 3.8 or above

Required Libraries:

- Flask
- NumPy
- Pandas
- Scikit-learn
- Joblib
- Requests (for Open-Meteo API integration)
- Matplotlib / Seaborn (for exploratory visualizations)

Development Tools:

- Jupyter Notebook or Google Colab (for data preprocessing, analysis, and model training)
- VS Code (for backend and frontend development)
- Git / GitHub (for version control)

Browser: Latest version of Chrome or Firefox (for viewing the Flask-based web interface)

10.2 Minimum Hardware requirements

- Processor: Dual-core 2.0 GHz or higher (Quad-core recommended for faster training)
- RAM: 4 GB minimum, 8 GB recommended for model training and smooth execution of Flask server
- Storage: At least 2 GB of free disk space for dataset, trained model files, logs, and project resources
- Display: Standard resolution (1366 × 768 or higher)
- Internet Connection:
Required for:
 - installing dependencies

- loading NASA POWER or Open-Meteo API data
- using Google Colab (optional)
- running API-based prediction system

11. REFERENCES

- Colucci R, Mahgoub I. Solar Irradiance Prediction with Ensemble Learning Method as Input for Battery Operation Optimization. In 2024 IEEE Texas Power and Energy Conference (TPEC) 2024 Feb 12 (pp. 1-6). IEEE. Available from: <https://doi.org/10.1109/TPEC60005.2024.10472247>
- Demir V. Evaluation of Solar Radiation Prediction Models Using AI: A Performance Comparison in the High-Potential Region of Konya, Türkiye. Atmosphere. 2025 Mar 30;16(4):398. Available from: <https://doi.org/10.3390/atmos16040398>
- Imam AA, Abusorrah A, Seedahmed MM, Marzband M. Accurate forecasting of global horizontal irradiance in Saudi Arabia: a comparative study of machine learning predictive models and feature selection techniques. Mathematics. 2024 Aug 22;12(16):2600. Available from: <https://doi.org/10.3390/math12162600>
- <https://www.geeksforgeeks.org/machine-learning/random-forest-algorithm-in-machine-learning/>
- <https://www.geeksforgeeks.org/machine-learning/hyperparameter-tuning/>